

Final Year Design Project(CC-411)

**TECHStile:QR Code-Based Textile Production and
Machine Management System**



Submitted by:

Irha	079879
Javeriya	079906
Aleesha	079912

Under the supervision of:

***Department of Information Technology
Faculty of Computing & Information Technology***

***University of the Punjab, Lahore Batch: 2022-2026 |
Semester: 7th***

DECLARATION

We hereby declare that this software, neither the whole nor a part, was copied from any source. It is further declared that we have developed this software and accompanying report based on our efforts. If any part of this project is proven to be copied from any source or found to be a reproduction of some other, we will stand by the consequences. No portion of the work presented has been submitted for any application for any other degree or qualification of this or any other university or institute of learning.

Signature: -----
Irha[079879]

Signature: -----
Javeriya[079906]

Signature: -----
Aleesha[079912]

CERTIFICATE OF APPROVAL

It is to certify that the final year design project (FYDP) of BSIT “**TECHStile: QR Code–Based Textile Production and Machine Management System**” was developed by **Irha (079879), Javeriya(079906) and Aleesha(079912)** Under the supervision of **PROF Marsad Akbar**, in my opinion it is fully adequate, in scope and qualifies for the degree of **Bachelors of Science in Information Technology**.

Signature: -----

FYDP Supervisor:

Signatures (Faculty Advisory Committee (FAC))

Signatures			
Name	Sir Marsad Akbar	Maam Nida	
	FAC1	FAC2	FAC3

Signatures: Faculty Advisory Committee (FAC)

Signature:----- Dated: _____

Executive Summary

QR Code-Based Textile Production and Machine Management System is made to help factories move from manual work to a digital system.

In this system, QR codes are placed on machines. Workers can scan these QR codes using a device, then enter production details and track fabric rolls easily. This helps in keeping accurate records of machine work and production. Managers and owners can also check machine usage, verify production, and approve payments using real time data.

The system is developed using a proper method with clear documents like SRS (Software Requirements Specification) and SDS (Software Design Specification). The system is designed using modern technologies and follows a client server structure with MVC design, which makes the system easy to manage and improve in the future.

The main goal of this project is to reduce mistakes, save time, and make factory work more transparent and efficient. Its success can be seen if production records become more accurate, manual work is reduced, and monitoring of workers and machines becomes easier.

In short, this QR Code-Based Textile Production and Machine Management System is a simple, smart, and low-cost solution for textile factories. It is designed to make production work easier and more organized by using a digital system instead of manual records.

The system helps in tracking machines, workers, and production data more accurately. It also allows managers to monitor work and make better decisions. Overall, it improves efficiency, reduces mistakes, and makes factory management easier.

Acknowledgment

We express our gratitude to our Almighty Allah, and after that, we are thankful to our families, who allowed us to get higher education, and we became capable of creating something productive. We're also thankful to our project advisor and department faculty for their support.

Signature: -----
Irha[079879]

Signature: -----
Javeriya[079906]

Signature: -----
Aleesha[079912]

Abbreviations

MVC	Model View Controller
SRS	System Requirements Specifications
API	Application Programming Interface
REST API	Representational State Transfer Application Programming Interface
IDE	Integrated Development Environment
UI/UX	User Interface / User Experience
UML	Unified Modeling Language
DFD	Data Flow Diagram
QR Code	Quick Response
TMFR	Textile Management Functional Requirements
MTBT	Mean Time Between Failures
TMUC	Textile Management Use Case
DCD	Design Class Diagram
HTTP	Hypertext Transfer
JSON	JavaScript Object Notation

Table of Contents

1. Introduction.....	13
1.1 Background.....	13
1.2 Problem Statement.....	14
1.3 Problem Solution.....	14
1.4 Objectives of the Proposed System.....	14
1.5 Scope.....	15
1.6 System Components.....	16
1.7 Related System Analysis.....	17
1.8 Vision Statement.....	17
1.9 System Limitations and Constraints.....	18
1.10 Tools and Technologies.....	18
1.11 Project Deliverables.....	19
1.12 Assumptions.....	19
1.13 Risks.....	19
1.14 Success Criteria.....	19
1.15 Gantt Chart.....	20
1.16 Summary.....	21
2. Analysis.....	23
2.1 User Classes and Characteristics.....	23
2.2 Requirement Identifying Technique.....	24
2.3 Functional Requirements.....	24
2.4 Non-Functional Requirements.....	31
2.4.1 Reliability.....	31
2.4.2 Usability.....	31
2.4.3 Performance.....	32
2.4.4 Security.....	32
2.5 External Interface Requirements.....	32
2.5.1 User Interface Requirements.....	32
2.5.2 Software Interfaces.....	32
2.5.3 Hardware Interfaces.....	32
2.5.4 Communication Interfaces.....	32
2.6 Use Case Analysis.....	33
2.6.1 Login.....	33
2.6.2 Role and Permission Setup.....	34
2.6.3 Add/Manage Users.....	35
2.6.4 Add/Manage Factory.....	36
2.6.5 Add/Manage Machines.....	37
2.6.6 Generate QR Code for Machine.....	38

2.6.7 Scan QR Code.....	39
2.6.8 Assign Employee to Machine.....	40
2.6.9 Employee Production Entry.....	41
2.6.10 Production Approval.....	42
2.6.11 Attendance Management.....	43
2.6.12 Payment Management.....	44
2.6.13 Notifications.....	45
2.6.14 Logout.....	46
2.7 Use Case Diagram.....	47
2.8 Summary.....	48
3. System Design.....	50
3.1 Design Considerations.....	50
3.1.1 Assumptions and Dependencies.....	50
3.1.2 Limitations.....	50
3.1.3 Risks.....	51
3.2 Design Models.....	52
3.2.1 Context Diagram.....	52
3.2.2 Class Diagram.....	53
3.2.3 Sequence Diagram.....	54
3.2.4 State Transition Diagram.....	61
3.3 Architecture Diagram.....	64
3.3.1 Architecture Diagram.....	64
3.3.2 Component Diagram.....	65
3.3.3 Layered Architecture (4-Tier).....	66
3.3.4 MVC.....	67
3.4 Data Design.....	67
3.4.1 QR Code Data Specification.....	67
3.4.2 ER Diagram.....	68
3.4.3 MySQL Data Structure.....	69
3.4.4 Data Processing and Organization.....	70
3.4.5 Data Dictionary.....	71
3.4.6 Object-Oriented (OO) Approach: Objects, Attributes, Methods & Method Parameters.....	73
3.4.7 User Interface Design.....	75
3. Screen Images:.....	75
3.5 Design Decisions.....	83
3.6 Summary.....	84

List of Tables

Table 1-1: System Analysis.....	16
Table 1-2: Tools and Technologies.....	17
Table 2-1: User classes and characteristics.....	22
Table 2-2: Description of FR-1.....	23
Table 2-3: Description of FR-2.....	24
Table 2-4: Description of FR-3.....	24
Table 2-5: Description of FR-4.....	25
Table 2-6: Description of FR-5.....	25
Table 2-7: Description of FR-6.....	26
Table 2-8: Description of FR-7.....	26
Table 2-9: Description of FR-8.....	27
Table 2-10: Description of FR-9.....	27
Table 2-11: Description of FR-10.....	28
Table 2-12: Description of FR-11.....	28
Table 2-13: Description of FR-12.....	29
Table 2-14: Description of FR-13.....	29
Table 2-15: Description of FR-14.....	30
Table 2-16: Use Case 1.....	32
Table 2-17: Use Case 2.....	33
Table 2-18: Use Case 3.....	34
Table 2-19: Use Case 4.....	35
Table 2-20: Use Case 5.....	36
Table 2-21: Use Case 6.....	37
Table 2-22: Use Case 7.....	38
Table 2-23: Use Case 8.....	39
Table 2-24: Use Case 9.....	40
Table 2-25: Use Case 10.....	41
Table 2-26: Use Case 11.....	42
Table 2-27: Use Case 12.....	43
Table 2-28: Use Case 13.....	44
Table 2-29: Use Case 14.....	45
Table 3-1: Risks.....	50
Table 3-2: Layered Architecture with Responsibilities.....	65
Table 3-3: MVC Component Modules Assigned.....	66
Table 3-4: QR Code Data Specification.....	67
Table 3-5: Table Data Dictionary.....	70

List Of Figures

Figure : Gantt Chart.....	19
Figure 1: Use Case Diagram.....	46
Figure 2: Context Diagram.....	51
Figure 3: Class Diagram.....	52
Figure 4: User Authentication.....	53
Figure 5: Scan Machine QR Code.....	54
Figure 6: Machine Assignment , Assign Shift.....	55
Figure 7: Add User, Assign Machines to employee.....	56
Figure 8: Sequence Diagram.....	57
Figure 9: View Production, Approve Production.....	58
Figure 10: View Payments, Approve Payments.....	59
Figure 11: Employee State Transaction Diagram.....	60
Figure 12: Owner State Sequence Diagram.....	61
Figure 13: Manager State Transaction Diagram.....	62
Figure 14: Architecture Diagram.....	63
Figure 15: Component Diagram.....	64
Figure 16 : ER Diagram.....	68

Chapter 1 Introduction

1. Introduction

TECHstile is a QR code based system that helps textile factories Owners to manage their machines, workers, production and payments in one place. Right now most small textile factories keep this information on paper registers, and this project replaces that paper work with a simple mobile application.

Our system lets the owner, manager and employees of a factory use one app to handle daily factory work. Every machine gets its own QR code, and this QR code is used to connect a worker to a machine, record production, and later calculate payment. Everything is stored in one database so owners and managers can check the current situation of the factory at any time instead of waiting for paper reports.

1.1 Background

Most textile factories in Pakistan, especially small and medium sized ones, still use manual registers to note down which worker is working on which machine, how much fabric was produced, and how much a worker should be paid. This method has a lot of problems. Registers get damaged, pages go missing, numbers get written wrong, and sometimes workers or managers write incorrect production amounts on purpose. Because of this, factory owners cannot trust the numbers fully and payments to workers are often delayed or disputed.

We wanted to build something that removes this paper work completely and gives the owner a live picture of what is happening on the factory floor, using nothing more than a phone camera and a QR code sticker on each machine.

1.2 Problem Statement

Textile factories in Pakistan face difficulty in tracking machine usage, worker output and payments because most of this work is still done manually on paper. This leads to wrong entries, lost records, and payments that do not match the actual work done. Owners and managers do not have a quick way to check what is happening in the factory without physically walking around and asking workers. Small factories cannot afford expensive ERP systems, so they are stuck

using paper or basic Excel sheets which still require manual entry and are just as easy to get wrong.

There is a need for a simple and low cost digital system that a small factory can actually use, without needing extra staff or training, to track machines, production and worker payments correctly.

1.3 Problem Solution

TECHstile solves this by giving every machine in the factory a unique QR code sticker. When a worker starts working on a machine, he scans the QR code and the app knows exactly which machine he is using. The worker then enters his production for the day through the app.

This production entry does not go directly into the system as final data. It first goes to the manager, who checks and approves or rejects it. After the manager approves it, the owner can also review it. Only after this checking process, the production is counted as verified and the worker's payment is calculated based on it. This two step checking removes the chance of someone entering wrong numbers and getting away with it, because both the manager and the owner are involved before any payment is finalized.

Owners and managers can log in any time and see which machines are being used, which employees are assigned where, and what has been produced so far, instead of waiting for someone to tell them.

1.4 Objectives of the Proposed System

1. To replace paper based factory registers with a mobile application.
2. To connect every machine to a unique QR code so workers and machines can be tracked correctly.
3. To let employees scan a QR code and enter their production directly from the factory floor.
4. To verify every production entry through a manager and owner before it is finalized.
5. To calculate employee payments based only on verified production, not on what the worker claims.

6. To give owners and managers real time visibility of machine usage, worker attendance and production, without needing to be physically present.
7. To give different access levels to owner, manager and employee so each person only sees and does what is relevant to their role.

1.5 Scope

TECHstile is a mobile application for Android that manages machines, employees, production and payments inside a textile factory. It is built to work with real factory operations, not just as a demo, and is currently being used with real data in an actual factory.

The scope of this project includes:

- Every machine in the factory will have its own unique QR code.
- Employees will be assigned to a machine by scanning its QR code.
- Employees will enter their daily production against the machine they scanned.
- Production entries will go through a two step verification, first by the manager and then by the owner.
- Employee attendance will be recorded and managed inside the app.
- Employee payments will be calculated based on verified production only.
- The system will support role based access, meaning owner, manager and employee each get different screens and permissions.
- Owners will be able to manage more than one factory, and manage machines and employees under each factory.
- Users will get notifications inside the app for important updates such as approvals or assignments.

1.6 System Components

1. Mobile Application A single Flutter based mobile application used by the owner, manager and employees. It provides QR scanning, production entry, approvals, attendance and payment screens based on the role of the logged in user.

2. QR Code Module Handles generating a unique QR code for every machine and reading that QR code through the phone camera when a user scans it.

3. Machine Management Module Lets the owner and manager add, update or remove machines, and keeps the current status and assignment of each machine.

4. Factory and User Management Module Lets the owner create and manage factories, and add managers and employees under each factory, along with their role and permissions.

5. Production Management Module Lets employees submit their daily production and lets managers and owners approve or reject each entry.

6. Attendance Module Keeps track of employee attendance so the factory has a digital record instead of a manual attendance register.

7. Payment Module Calculates and stores employee payments based on production that has already been verified by the manager and owner.

8. Notification Module Sends in-app notifications to users when something needs their attention, such as a production entry waiting for approval or a new assignment.

9. Backend and Database A Laravel based backend with a MySQL database that stores all machine, user, production, attendance and payment data, and connects to the mobile app through REST APIs.

1.7 Related System Analysis

Table 1-1: System Analysis

Application Name	Weakness	TECHstile Solution
Manual Register System	Paper records that can be lost, damaged or written incorrectly. No real time visibility of production or payments.	Replaces paper with a digital app where every production entry is linked to a specific machine and worker.
Excel Based Factory Records	Data is entered manually and can be changed or deleted by anyone with access to the file. No link between machine, worker and output.	Every entry is tied to a QR scanned machine and a logged in employee, so entries cannot be added without going through the app.
Basic Payroll Software	Calculates salary but does not check whether the reported production is actually correct.	Payments are only calculated after the manager and owner have verified the production.
Generic ERP Systems	Expensive and complicated, mostly built for large factories with dedicated IT staff.	TECHstile is simple, mobile based, and made for small and medium sized textile factories that cannot afford a full ERP system.

1.8 Vision Statement

For textile factory owners, managers and workers who want an easier way to manage machines, production and payments, TECHstile is a mobile application that replaces manual registers with QR code based tracking. Unlike paper registers or Excel sheets, TECHstile links every machine, worker and production entry together and verifies data before it is used for payments, giving factory owners a system they can actually trust.

1.9 System Limitations and Constraints

1. The application currently works on Android phones only.
2. QR scanning depends on the phone camera, so a low quality camera or a damaged QR code sticker can cause scanning issues.
3. The system needs an active internet connection to work, since production entry, approvals and payments all go through the server. There is no offline mode at this stage.
4. The system is built and tested for factories similar in size and workflow to the one it is currently deployed in, and may need adjustments for very different factory setups.

1.10 Tools and Technologies

Table 1-2: Tools and Technologies

Category	Tool / Version	Purpose
Programming Language	Dart (Flutter SDK)	Used to build the mobile application
Mobile Framework	Flutter 3.x	Used for building the Android app
Backend Framework	PHP Laravel 10.x	Handles server side logic and APIs
Database	MySQL 8.x	Stores users, machines, production and payment data
API Type	REST API (JSON)	Connects the mobile app with the backend
QR Code Library	Flutter QR / Mobile Scanner	Used for generating and scanning QR codes
IDE	Visual Studio Code	Used for both frontend and backend development
UI Design	Flutter Widgets	Used to design the app screens

1.11 Project Deliverables

Project Proposal

Software Requirement Specification (SRS)

Software Design Specification (SDS)

Prototype

Comprehensive Evaluation Document (Proposal, SRS, SDS)

1.12 Assumptions

- Owner, manager and employees have access to Android smartphones with a working camera.
- Every machine has a physical QR code sticker properly attached and undamaged.
- A stable internet connection is available at the factory for daily use.
- Employees enter their production data honestly and check it before submitting.
- Manager and owner are available on a regular basis to review and approve production entries

1.13 Risks

- QR code sticker may get damaged, faded, or removed from the machine.
- Employees may enter wrong or exaggerated production quantities.
- Weak or unstable internet connection may delay production entry, approval, or payment calculation.
- Delay by the manager or owner in reviewing entries may delay employee payments.
- Server or database issues may affect access to real-time factory data.

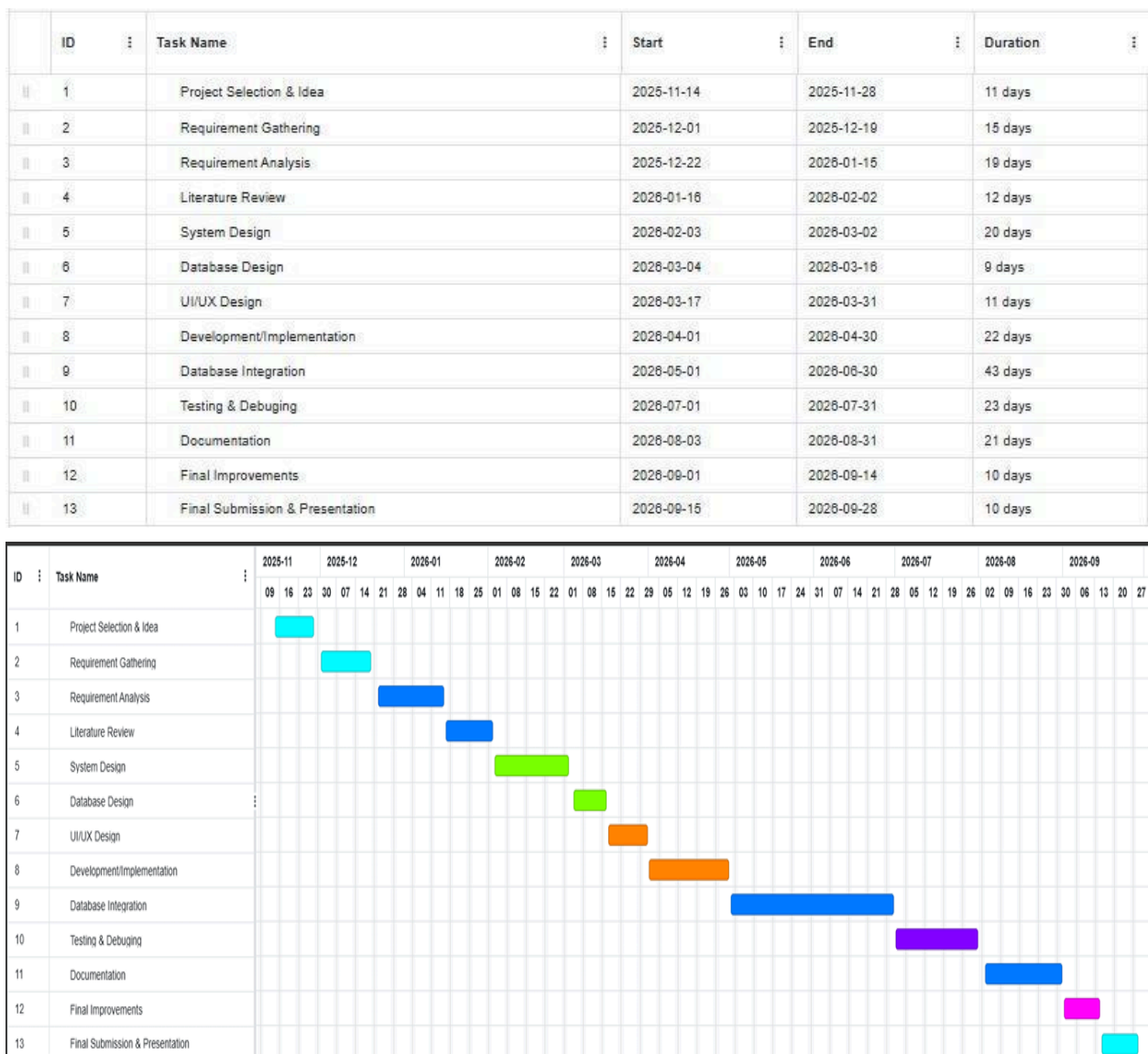
1.14 Success Criteria

- Every machine in the factory is successfully linked to a working, scannable QR code.
- Employees are able to scan, get assigned, and submit daily production without difficulty.
- Every production entry goes through proper two step verification (manager and owner) before approval.

- Employee payments are calculated only from verified/approved production, with no disputes.
- Owner and manager can view real-time visibility of machines, attendance, and production at any time.
- The system is successfully deployed and used with real data inside an actual textile factory.

1.15 Gantt Chart

Figure : Gantt Chart



1.16 Summary

This chapter explained why TECHstile was built and what problem it solves. Textile factories, especially small ones, still depend on paper registers which cause errors and delays in tracking production and payments. TECHstile solves this by giving every machine a QR code and building a mobile app around it, where production is entered, checked by a manager and owner, and only then used to calculate payment. This chapter also covered the objectives, scope, main components, tools used, and the overall plan of the project. The next chapters go into the detailed requirements and design of the system.

Chapter 2 Requirements Analysis

2. Analysis

This chapter looks at how the different users of TECHstile interact with the system, and defines what the system must do to support their work. We studied the actual factory workflow, including how a worker is assigned to a machine, how his production is checked, and how his payment is calculated, and used this to write out the functional and non-functional requirements. This chapter also defines the user classes and explains their responsibilities, and uses use cases to show how each type of user interacts with the system.

2.1 User Classes and Characteristics

TECHstile has three user classes: Owner, Manager and Employee.

Table 2-1: User classes and characteristics

User Class	Characteristics
Owner	The Owner has full access to the system. The owner can add and manage factories, add machines, generate QR codes, add managers and employees, give final approval on production and payments, and view the complete activity of the factory.
Manager	The Manager handles the daily operations of a factory. The manager assigns employees to machines by scanning QR codes, checks the production entered by employees, and monitors machine status and worker performance.
Employee	The Employee is the worker who uses the app mainly to scan a machine QR code and enter his daily production. The employee can also mark attendance and view his own payment history.

2.2 Requirement Identifying Technique

TECHstile is a role based mobile application used by an owner, a manager and employees inside a factory. Because the interaction between these users follows a clear workflow, such as scanning a QR code, assigning a machine, entering production, and approving payment, we used the Use Case Technique as our main method to identify requirements.

A Use Case Diagram shows the interaction between the different users and the system, and highlights the main functions of the system. A Use Case Analysis then looks at each use case in detail to identify exact behavior of system and make sure every requirement is captured correctly.

2.3 Functional Requirements

This section lists the functional requirements of TECHstile. Each requirement is based on a feature that is already built and working in the current version of the system.

Table 2-2: Description of FR-1

Attribute	Description
Identifier	TMFR-1
Title	Login System
Requirement	The system shall allow users (Owner, Manager, Employee) to log in using their email and password.
Source	System Requirement, Factory Owner
Rationale	To make sure only registered users can access the system, and that each user only sees the features allowed for his role.
Business Rule	Only registered users can log in. Each user must have a unique email. Passwords are stored in encrypted form.
Dependencies	User account must already exist in the system.
Priority	High

Table 2-3: Description of FR-2

Attribute	Description
Identifier	TMFR-2
Title	Role and Permission Setup
Requirement	The system shall allow the owner to create roles and set permissions for each role, deciding which actions a manager or employee is allowed to perform.
Source	System Requirement, Factory Owner
Rationale	To control what each type of user can see and do inside the app, based on his responsibility in the factory.
Business Rule	Only the owner can create or edit roles and permissions. A user can be assigned only one role at a time.
Dependencies	User must be logged in as owner.
Priority	High

Table 2-4: Description of FR-3

Attribute	Description
Identifier	TMFR-3
Title	Add/Manage Users
Requirement	The system shall allow the owner to add, update, or remove manager and employee accounts, and assign a role to each of them.
Source	System Requirement, Factory Owner
Rationale	To let the owner control who is working in the factory and what access they have inside the app.
Business Rule	Each user must have a unique email. Only the owner can manage user accounts.
Dependencies	Login System (TMFR-1), Role and Permission Setup (TMFR-2).
Priority	High

Table 2-5: Description of FR-4

Attribute	Description
Identifier	TMFR-4
Title	Add/Manage Factory
Requirement	The system shall allow the owner to add a new factory and manage its details, and switch between factories if he owns more than one.
Source	System Requirement, Factory Owner
Rationale	To let one owner manage multiple factories from a single account.
Business Rule	Only the owner can add or remove a factory. Every machine and user belongs to one specific factory.
Dependencies	Login System (TMFR-1).
Priority	High

Table 2-6: Description of FR-5

Attribute	Description
Identifier	TMFR-5
Title	Add/Manage Machines
Requirement	The system shall allow the owner to register a new machine by entering its details, and to update or remove machine records.
Source	System Requirement, Factory Owner
Rationale	To keep a proper digital record of every machine in the factory.
Business Rule	Each machine must have a unique ID. Only the owner can add or update machine records.
Dependencies	Add/Manage Factory (TMFR-4).
Priority	High

Table 2-7: Description of FR-6

Attribute	Description
Identifier	TMFR-6
Title	Generate QR Code for Machine
Requirement	The system shall generate a unique QR code for every registered machine, which can be downloaded and attached to the physical machine.
Source	System Requirement, Factory Owner
Rationale	To give every machine a quick and reliable way to be identified during assignment and production entry.
Business Rule	A QR code is linked to only one machine at a time.
Dependencies	Add/Manage Machines (TMFR-5).
Priority	High

Table 2-8: Description of FR-7

Attribute	Description
Identifier	TMFR-7
Title	Scan QR Code
Requirement	The system shall allow owner, manager and employee to scan a machine QR code using the phone camera to view or use machine details.
Source	System Requirement, Owner, Manager, Employee
Rationale	To let users quickly identify a machine without typing anything manually.
Business Rule	Only a valid, already registered QR code will be accepted by the system.
Dependencies	Generate QR Code for Machine (TMFR-6).
Priority	High

Table 2-9: Description of FR-8

Attribute	Description
Identifier	TMFR-8
Title	Assign Employee to Machine
Requirement	The system shall allow the manager or owner to assign an employee to a machine after scanning its QR code.
Source	System Requirement, Owner, Manager
Rationale	To link a specific worker with a specific machine so production can be tracked correctly.
Business Rule	A machine can be assigned to one employee at a time.
Dependencies	Scan QR Code (TMFR-7), Login System (TMFR-1).
Priority	High

Table 2-10: Description of FR-9

Attribute	Description
Identifier	TMFR-9
Title	Employee Production Entry
Requirement	The system shall allow an employee to enter his daily production against the machine he is assigned to.
Source	System Requirement, Employee
Rationale	To record machine wise and employee wise production accurately, and support better tracking of daily output.
Business Rule	A production entry must be linked to a valid machine and a valid employee. Production quantity must be greater than zero.
Dependencies	Assign Employee to Machine (TMFR-8).
Priority	High

Table 2-11: Description of FR-10

Attribute	Description
Identifier	TMFR-10
Title	Production Approval
Requirement	The system shall allow the manager to review and approve or reject the production entered by an employee, and allow the owner to give a final approval after the manager has approved it.
Source	System Requirement, Owner, Manager
Rationale	To make sure production data is correct and genuine before it is used to calculate payment.
Business Rule	Only a manager can give the first approval. Only the owner can give the final approval. Once approved, an entry cannot be changed without proper authorization.
Dependencies	Employee Production Entry (TMFR-9).
Priority	High

Table 2-12: Description of FR-11

Attribute	Description
Identifier	TMFR-11
Title	Attendance Management
Requirement	The system shall allow employee attendance to be marked and viewed inside the app, and allow the manager or owner to review it.
Source	System Requirement, Owner, Manager, Employee
Rationale	To keep a digital attendance record instead of a manual attendance register.
Business Rule	Attendance must be linked to a valid employee and a valid date.
Dependencies	Login System (TMFR-1).
Priority	Medium

Table 2-13: Description of FR-12

Attribute	Description
Identifier	TMFR-12
Title	Payment Management
Requirement	The system shall calculate an employee's payment based only on his approved production, and allow the manager or owner to view, add, or update payment records.
Source	System Requirement, Owner, Manager
Rationale	To make sure payments are based only on checked and approved work, reducing manual error and disagreement over wages.
Business Rule	Payment can only be calculated from approved production records.
Dependencies	Production Approval (TMFR-10).
Priority	High

Table 2-14: Description of FR-13

Attribute	Description
Identifier	TMFR-13
Title	Notifications
Requirement	The system shall send an in app notification to a user when an action needs his attention, such as a production entry waiting for approval.
Source	System Requirement, Owner, Manager, Employee
Rationale	To keep users updated about important activity without needing to check the app manually every time.
Business Rule	A notification must be linked to the specific user it is meant for.
Dependencies	Login System (TMFR-1).
Priority	Medium

Table 2-15: Description of FR-14

Attribute	Description
Identifier	TMFR-14
Title	Logout
Requirement	The system shall allow a logged in user to log out and end his active session.
Source	System Requirement, Owner, Manager, Employee
Rationale	To keep the system secure by making sure no one can use the app after the actual user has finished.
Business Rule	Only a logged in user can log out. After logout, the user must log in again to use the app.
Dependencies	Login System (TMFR-1).
Priority	Medium

2.4 Non-Functional Requirements

This section describes the quality attributes of TECHstile, meaning how well the system should perform rather than what it should do.

2.4.1 Reliability

The system should not lose any approved production or payment data, even if the app crashes or the phone loses power in the middle of an action. Once a record is saved and approved, it should stay safe in the database.

2.4.2 Usability

A new employee should be able to learn how to scan a QR code and enter production within a few minutes of using the app for the first time. Error messages should be simple and clear, not technical. Each user should only see the screens and options relevant to his role, so the app does not feel confusing.

2.4.3 Performance

Machine details should appear quickly, within a few seconds, after a QR code is scanned. A production entry should be saved without any noticeable delay. The app should work smoothly with normal factory-level usage without slowing down.

2.4.4 Security

Only logged in and authorized users can view or change data based on their role. Passwords are stored in encrypted form, and data sent between the app and the server is transmitted securely.

2.5 External Interface Requirements

2.5.1 User Interface Requirements

TECHstile provides a simple and consistent interface for owner, manager and employee. The screens use clear buttons, labels and menus, and every screen a user sees is based only on what his role is allowed to access. Error and confirmation messages follow the same style across the app so users are not confused.

2.5.2 Software Interfaces

The mobile app communicates with the backend through REST APIs. The backend is built using Laravel, and all data is stored in a MySQL database.

2.5.3 Hardware Interfaces

The app runs on Android smartphones. It uses the phone camera to scan machine QR codes. No extra or special hardware is needed to run the system.

2.5.4 Communication Interfaces

The app needs an active internet connection to send and receive data from the server. Data is sent using secure HTTPS requests. Users are informed about important updates, such as an approval or assignment, through in app notifications.

2.6 Use Case Analysis

This section explains each use case of TECHstile, matching every functional requirement listed in section 2.3.

2.6.1 Login

Table 2-16: Use Case 1

Field	Description
UC Identifier	TM-UC1
UC Name	Login
Requirement Traceability	TMFR-1
Purpose	To let owner, manager and employee log in and reach their role based dashboard.
Priority	High
Preconditions	The user must already have an account in the system.
Post Conditions	The user is logged in and taken to his dashboard.
Actors	Owner, Manager, Employee
Main Success Scenario	1. User opens the login screen. 2. User enters email and password. 3. System checks the credentials. 4. System identifies the user's role. 5. System opens the correct dashboard for that role.
Alternate Flow	If the email or password is wrong, the system shows an error and keeps the user on the login screen.

2.6.2 Role and Permission Setup

Table 2-17: Use Case 2

Field	Description
UC Identifier	TM-UC2
UC Name	Set Role and Permission
Requirement Traceability	TMFR-2
Purpose	To let the owner create a role and decide what actions that role is allowed to perform.
Priority	High
Preconditions	The owner must be logged in.
Post Conditions	A role is created or updated with a set of permissions.
Actors	Owner
Main Success Scenario	1. Owner opens the role management screen. 2. Owner creates a new role or selects an existing one. 3. Owner selects the permissions for that role. 4. System saves the role and its permissions.
Alternate Flow	If the owner tries to remove a role that is still assigned to a user, the system asks for confirmation first.

2.6.3 Add/Manage Users

Table 2-18: Use Case 3

Field	Description
UC Identifier	TM-UC3
UC Name	Manage Users
Requirement Traceability	TMFR-3
Purpose	To let the owner add a manager or employee account and assign a role to them.
Priority	High
Preconditions	The owner must be logged in.
Post Conditions	The user account is created, updated, or removed.
Actors	Owner
Main Success Scenario	1. Owner opens the user management screen. 2. Owner enters the new user's details and selects a role. 3. System creates the account. 4. The new user can now log in with those credentials.
Alternate Flow	If the entered email is already registered, the system shows an error and does not create a duplicate account.

2.6.4 Add/Manage Factory

Table 2-19: Use Case 4

Field	Description
UC Identifier	TM-UC4
UC Name	Manage Factory
Requirement Traceability	TMFR-4
Purpose	To let the owner add a factory and manage its details.
Priority	High
Preconditions	The owner must be logged in.
Post Conditions	The factory is added, updated, or removed from the system.
Actors	Owner
Main Success Scenario	1. Owner opens the factory management screen. 2. Owner enters the factory details. 3. System saves the new factory. 4. Owner can now add machines and users under this factory.
Alternate Flow	If required details are missing, the system shows an error and does not save the factory.

2.6.5 Add/Manage Machines

Table 2-20: Use Case 5

Field	Description
UC Identifier	TM-UC5
UC Name	Manage Machines
Requirement Traceability	TMFR-5
Purpose	To let the owner register a machine and keep its record updated.
Priority	High
Preconditions	The owner must be logged in and a factory must already exist.
Post Conditions	The machine record is created, updated, or removed.
Actors	Owner
Main Success Scenario	1. Owner opens the machine management screen. 2. Owner enters machine details such as name, type and model. 3. System saves the machine with a unique ID.
Alternate Flow	If the machine ID already exists, the system shows an error and asks the owner to check the details.

2.6.6 Generate QR Code for Machine

Table 2-21: Use Case 6

Field	Description
UC Identifier	TM-UC6
UC Name	Generate QR Code
Requirement Traceability	TMFR-6
Purpose	To create a unique QR code for a registered machine.
Priority	High
Preconditions	The machine must already be registered, and the user must be logged in as owner.
Post Conditions	A QR code is generated and linked to the machine.
Actors	Owner
Main Success Scenario	1. Owner opens the machine details. 2. Owner selects generate QR code. 3. System creates a unique QR code for that machine. 4. Owner can download or share the QR code.
Alternate Flow	If the machine already has a QR code, the system shows the existing one instead of making a new one.

2.6.7 Scan QR Code

Table 2-22: Use Case 7

Field	Description
UC Identifier	TM-UC7
UC Name	Scan QR Code
Requirement Traceability	TMFR-7
Purpose	To let a user scan a machine's QR code and view its details.
Priority	High
Preconditions	The machine must already have a QR code, and the user must be logged in.
Post Conditions	The system displays the details of the scanned machine.
Actors	Owner, Manager, Employee
Main Success Scenario	1. User opens the scan option. 2. User points the camera at the machine QR code. 3. System reads the QR code. 4. System shows the machine details on screen.
Alternate Flow	If the QR code is invalid or damaged, the system shows an error and asks the user to scan again.

2.6.8 Assign Employee to Machine

Table 2-23: Use Case 8

Field	Description
UC Identifier	TM-UC8
UC Name	Assign Employee to Machine
Requirement Traceability	TMFR-8
Purpose	To connect a specific employee with a specific machine using its QR code.
Priority	High
Preconditions	The manager or owner must be logged in, and the machine must have a QR code.
Post Conditions	The employee is assigned to the scanned machine.
Actors	Manager, Owner
Main Success Scenario	1. Manager scans the machine QR code. 2. System shows machine details. 3. Manager selects an employee to assign. 4. System saves the assignment.
Alternate Flow	If the machine is already assigned to another employee, the system asks for confirmation before changing it.

2.6.9 Employee Production Entry

Table 2-24: Use Case 9

Field	Description
UC Identifier	TM-UC9
UC Name	Enter Production
Requirement Traceability	TMFR-9
Purpose	To let an employee submit his daily production against his assigned machine.
Priority	High
Preconditions	The employee must already be assigned to a machine.
Post Conditions	The production entry is saved and marked as pending approval.
Actors	Employee
Main Success Scenario	1. Employee scans his assigned machine. 2. Employee enters the production quantity for the day. 3. System saves the entry as pending.
Alternate Flow	If the employee enters an invalid quantity, such as zero or a negative number, the system shows an error and does not save the entry.

2.6.10 Production Approval

Table 2-25: Use Case 10

Field	Description
UC Identifier	TM-UC10
UC Name	Approve Production
Requirement Traceability	TMFR-10
Purpose	To let the manager and owner check a production entry before it is used for payment.
Priority	High
Preconditions	A production entry must already exist and be pending.
Post Conditions	The production entry is marked as approved or rejected.
Actors	Manager, Owner
Main Success Scenario	1. Manager opens the pending production entries. 2. Manager reviews and approves an entry. 3. System sends the entry to the owner. 4. Owner reviews and gives final approval.
Alternate Flow	If the manager or owner rejects the entry, the system marks it as rejected and notifies the employee.

2.6.11 Attendance Management

Table 2-26: Use Case 11

Field	Description
UC Identifier	TM-UC11
UC Name	Mark Attendance
Requirement Traceability	TMFR-11
Purpose	To record an employee's daily attendance inside the app.
Priority	Medium
Preconditions	The employee must be logged in.
Post Conditions	The attendance record is saved for that date.
Actors	Employee, Manager, Owner
Main Success Scenario	1. Employee opens the attendance option. 2. Employee marks himself present. 3. System saves the attendance record with date and time. 4. Manager or owner can view the attendance later.
Alternate Flow	If attendance for that day is already marked, the system shows the existing status instead of creating a duplicate.

2.6.12 Payment Management

Table 2-27: Use Case 12

Field	Description
UC Identifier	TM-UC12
UC Name	Manage Payment
Requirement Traceability	TMFR-12
Purpose	To calculate and manage an employee's payment based on his approved production.
Priority	High
Preconditions	The employee must have at least one approved production entry.
Post Conditions	The payment amount is calculated and stored against the employee.
Actors	Manager, Owner
Main Success Scenario	1. Manager opens the payment section. 2. System calculates the amount using the employee's approved production. 3. Manager or owner reviews and saves the payment.
Alternate Flow	If there is no approved production for the employee, the system shows that no payment can be calculated yet.

2.6.13 Notifications

Table 2-28: Use Case 13

Field	Description
UC Identifier	TM-UC13
UC Name	Receive Notification
Requirement Traceability	TMFR-13
Purpose	To alert a user inside the app when something needs his attention.
Priority	Medium
Preconditions	The user must be logged in.
Post Conditions	The user sees the notification and can open it to view details.
Actors	Owner, Manager, Employee
Main Success Scenario	1. An action happens that needs the user's attention, such as a production entry waiting for approval. 2. System creates a notification for the relevant user. 3. User opens the notification and is taken to the related screen.
Alternate Flow	If the user has no new notifications, the system shows an empty notification list.

2.6.14 Logout

Table 2-29: Use Case 14

Field	Description
UC Identifier	TM-UC14
UC Name	Logout
Requirement Traceability	TMFR-14
Purpose	To let a logged in user end his session securely.
Priority	Medium
Preconditions	The user must already be logged in.
Post Conditions	The user's session is ended and he is returned to the login screen.
Actors	Owner, Manager, Employee
Main Success Scenario	1. User selects the logout option. 2. System ends the active session. 3. System returns the user to the login screen.
Alternate Flow	None.

2.7 Use Case Diagram

Figure 1: Use Case Diagram



2.8 Summary

This chapter explained how TECHstile was analyzed before it was built. We defined the three user classes, owner, manager and employee, and listed the fourteen functional requirements that the current system actually supports, from login to logout. We also covered the non-functional requirements, the interfaces the system depends on, and a few key use cases that show how the main features work together. The next chapter covers the design of the system based on these requirements.

Chapter 3 Design and Architecture

3. System Design

This chapter describes the structure of TECHstile and how its different parts work together. The system works as a mobile based client server application, where Android devices used by the owner, manager and employees communicate with a central backend server and database.

3.1 Design Considerations

3.1.1 Assumptions and Dependencies

- The system assumes that owner, manager and employees actively use the mobile app for their daily work.
- A stable internet connection is needed to scan QR codes, update machine status, and send production data to the server.
- Every machine is assumed to have its own unique QR code physically attached to it.
- The system depends on employees entering correct production data, and on the manager and owner checking it properly and on time.
- Secure login is assumed to be available so that owner, manager and employee only get access according to their role.

3.1.2 Limitations

- QR code scanning and data updates may not work properly in areas with weak or unstable internet.
- The system depends on manual data entry by employees, which can lead to mistakes if not checked properly.
- Payment approval depends on the manager and owner checking production on time, so any delay on their part can delay the payment as well.

3.1.3 Risks

Table 3-1: Risks

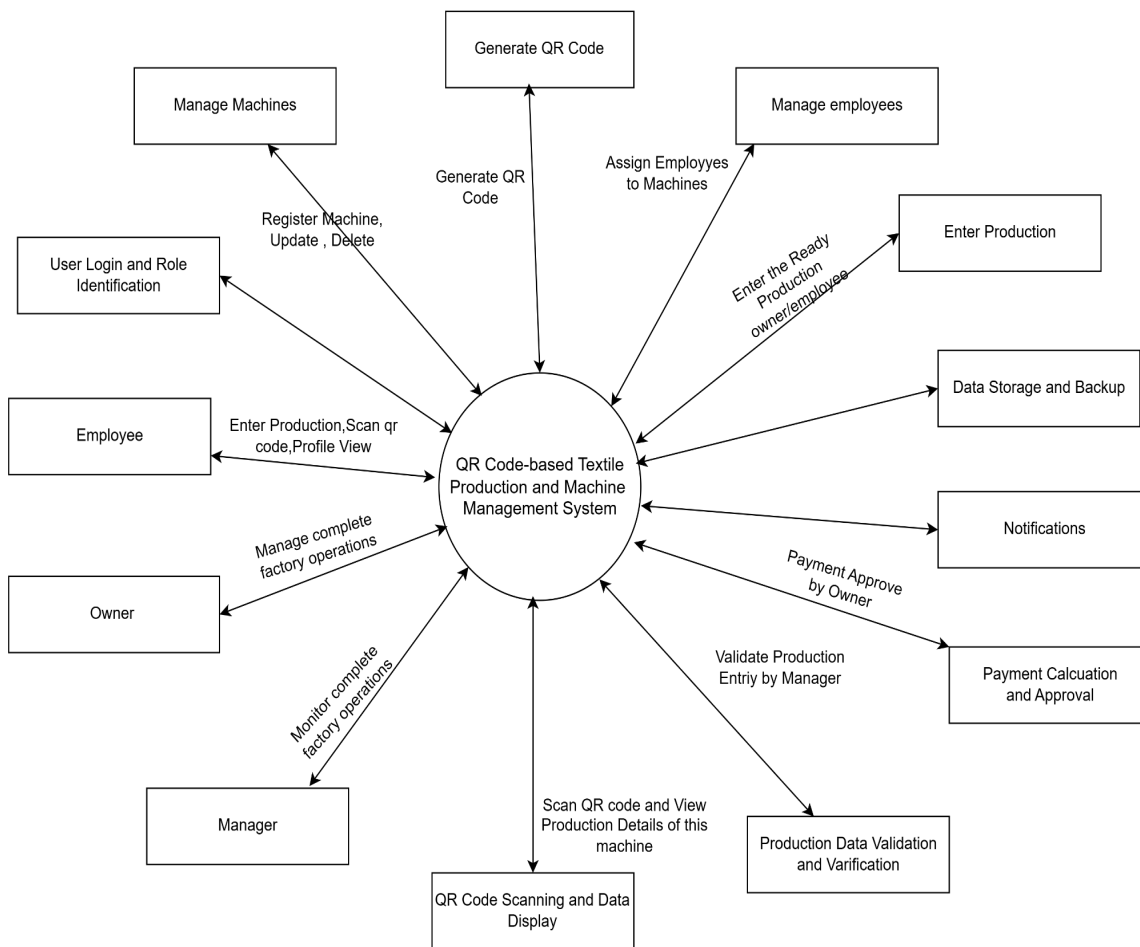
Risk	Design Mitigation
QR code gets damaged	The owner can generate a new QR code for the same machine.
Wrong data entered by employee	Manager and owner verification before the entry is approved.
Internet connection fails	The user is shown a clear message and can resubmit once the connection is back.
Server or database issue	Regular database backups are maintained on the server side.

3.2 Design Models

A design model is a structured representation of the components, relationships, and processes involved in creating the system. The design model views requirements specifications and use cases for a system and describes system implementation.

3.2.1 Context Diagram

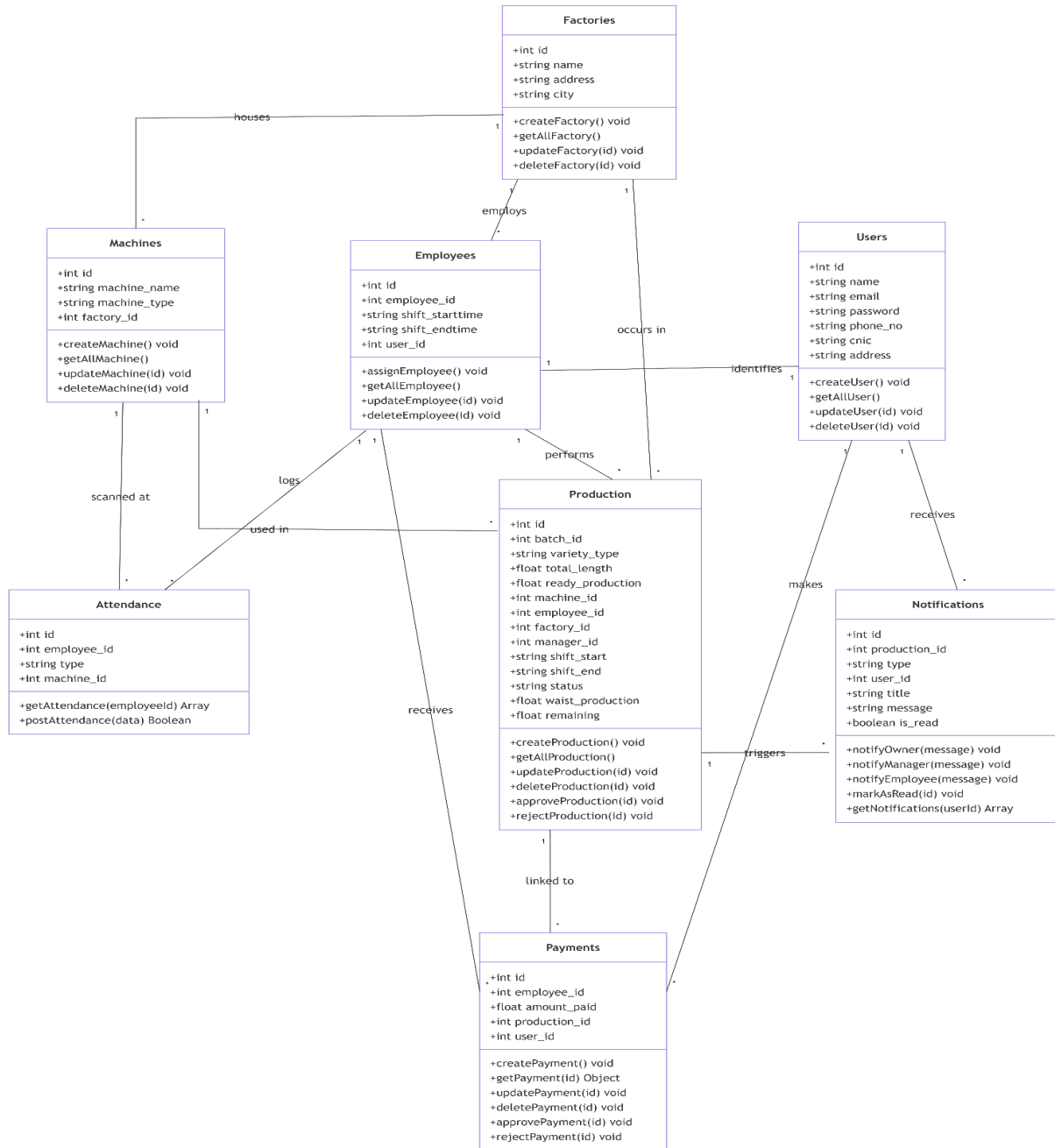
Figure 2: Context Diagram



3.2.2 Class Diagram

The class diagram shows the main entities of the system, such as User, Machine, Factory, Production, Attendance, Payment and Notification, along with their attributes and how these entities are connected to each other.

Figure 3: Class Diagram

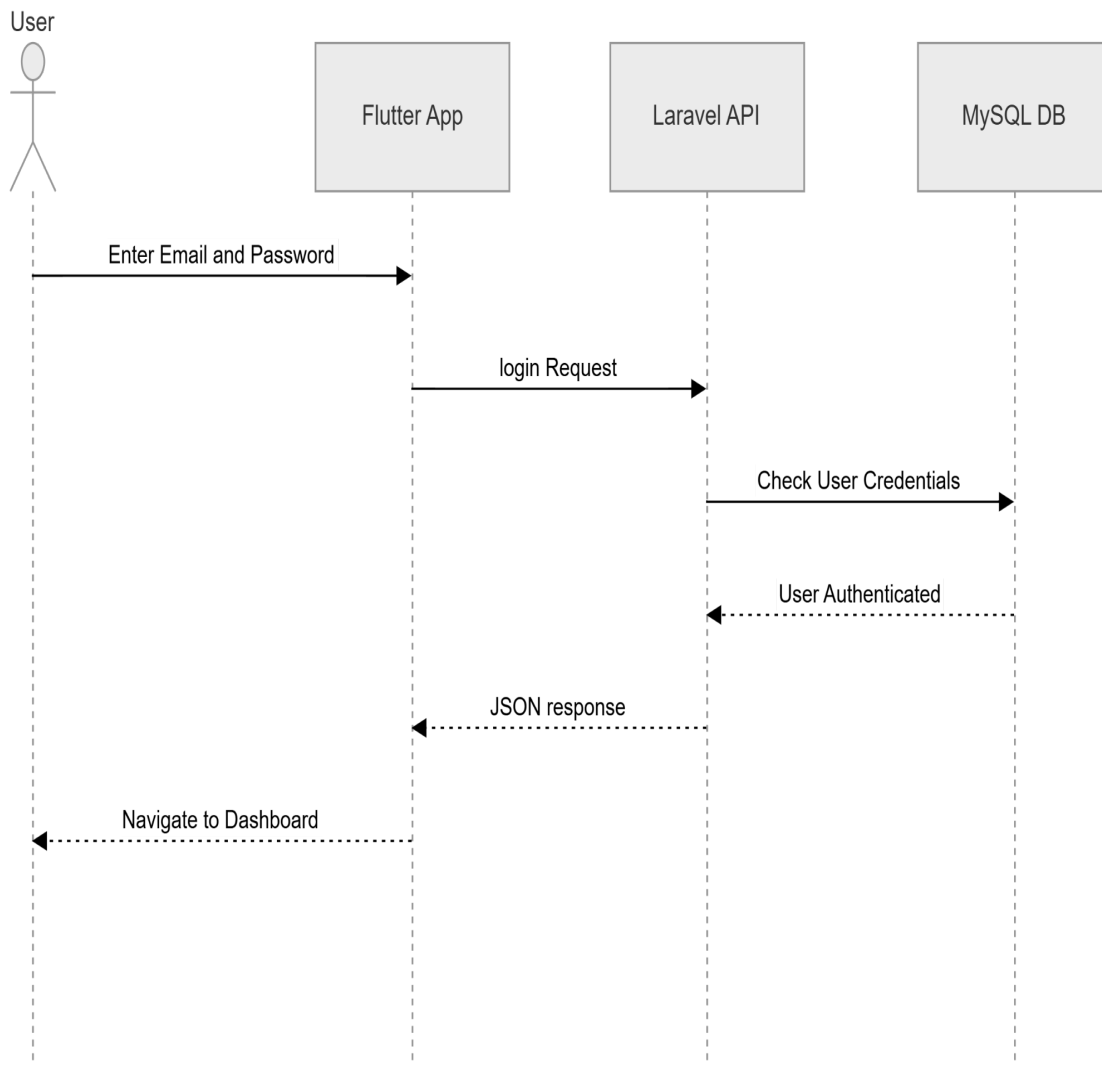


3.2.3 Sequence Diagram

The sequence diagram shows the step by step flow of one of the main processes of the system, such as an employee scanning a machine, entering production, and that production going through manager and owner approval.

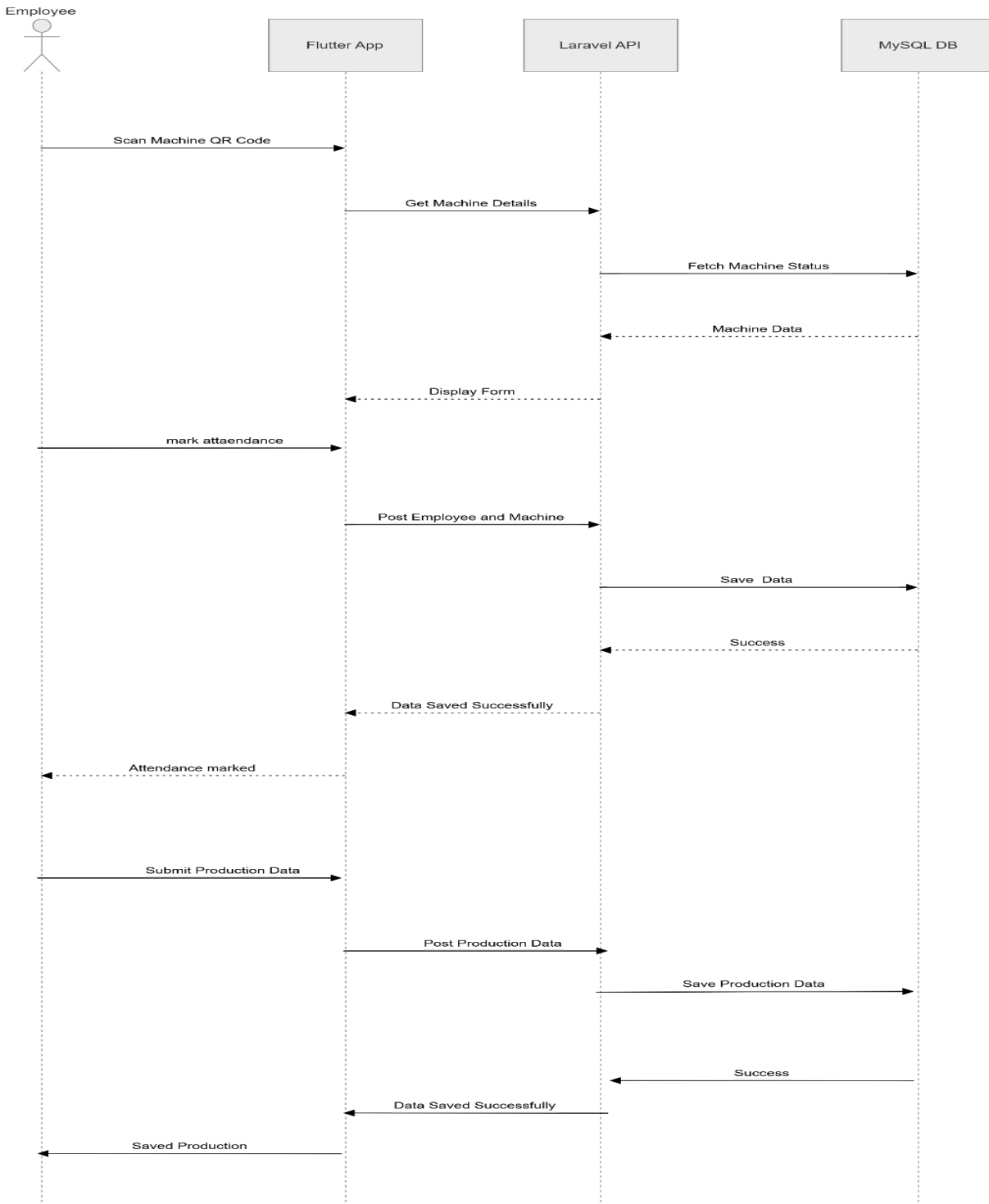
Figure 4: User Authentication

User Authentication:



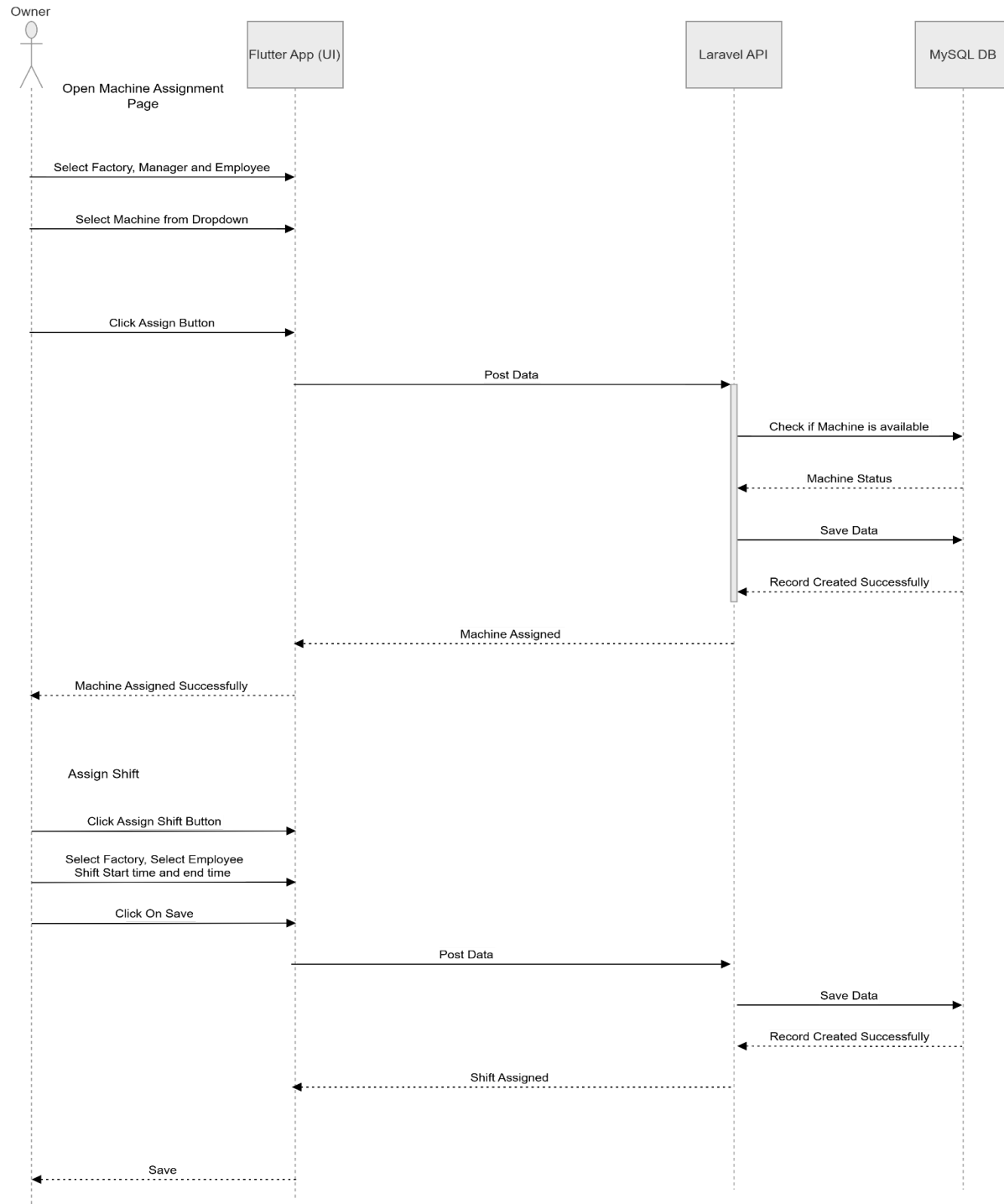
Scan Machine QR Code, Mark Attendance, Enter Production:

Figure 5: Scan Machine QR Code



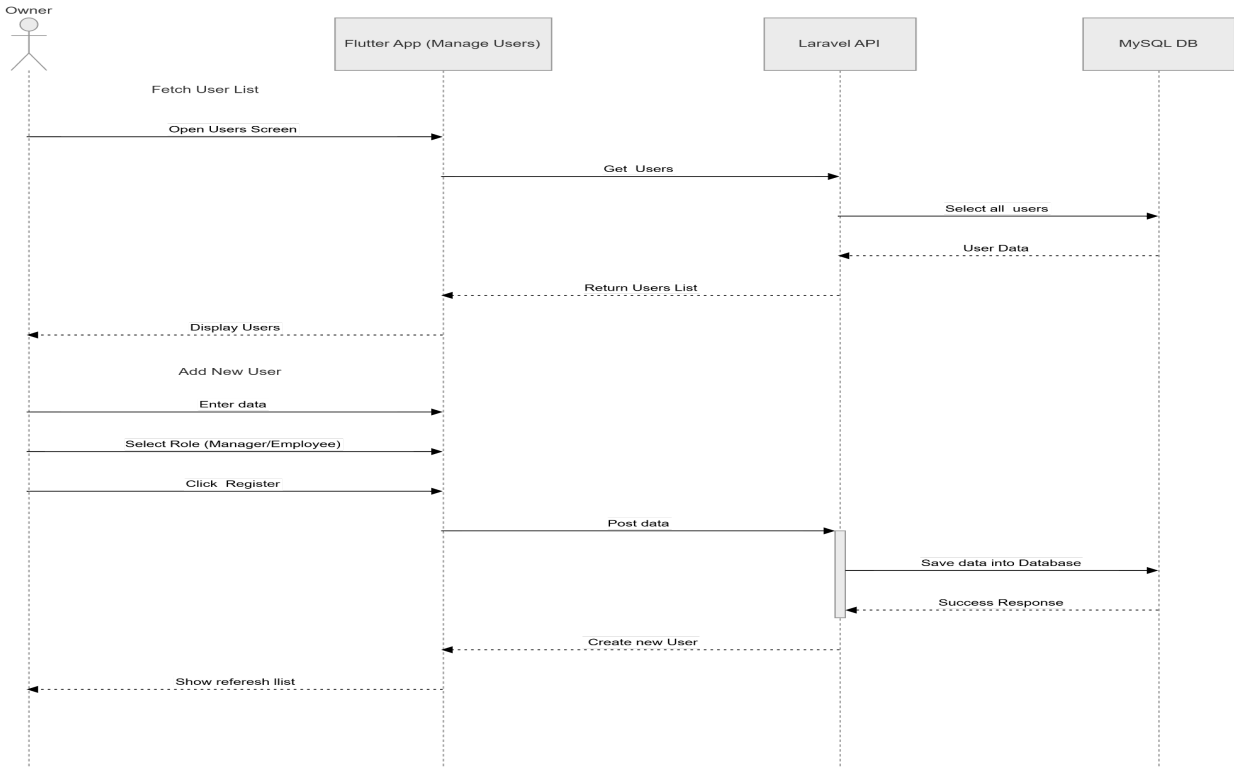
Machine Assignment, Assign Shift:

Figure 6: Machine Assignment , Assign Shift



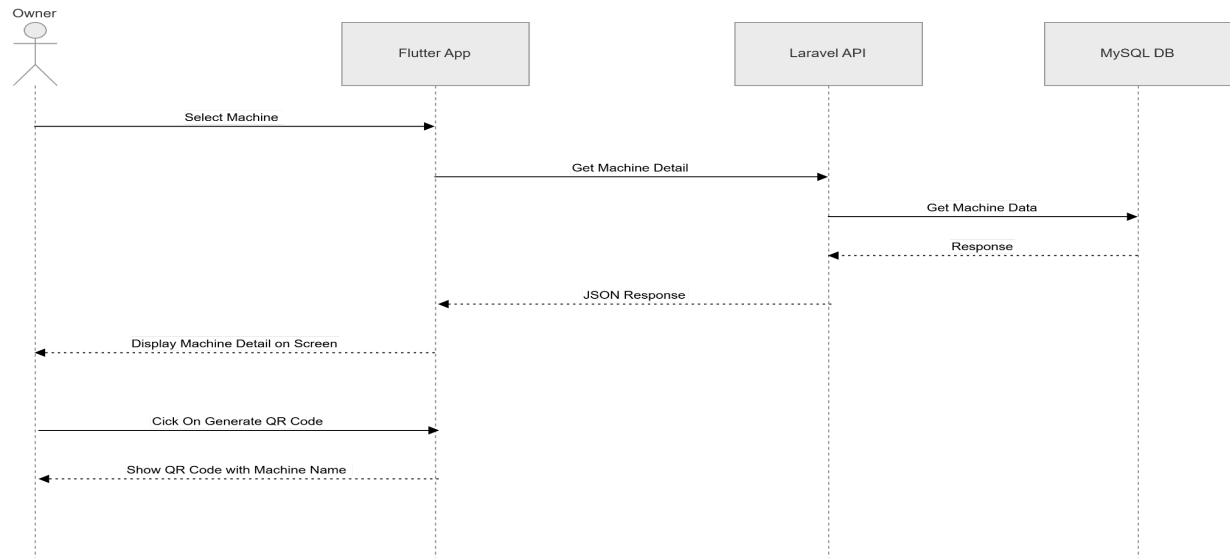
Add Users Assign Machines to Employees:

Figure 7: Add User, Assign Machines to employee



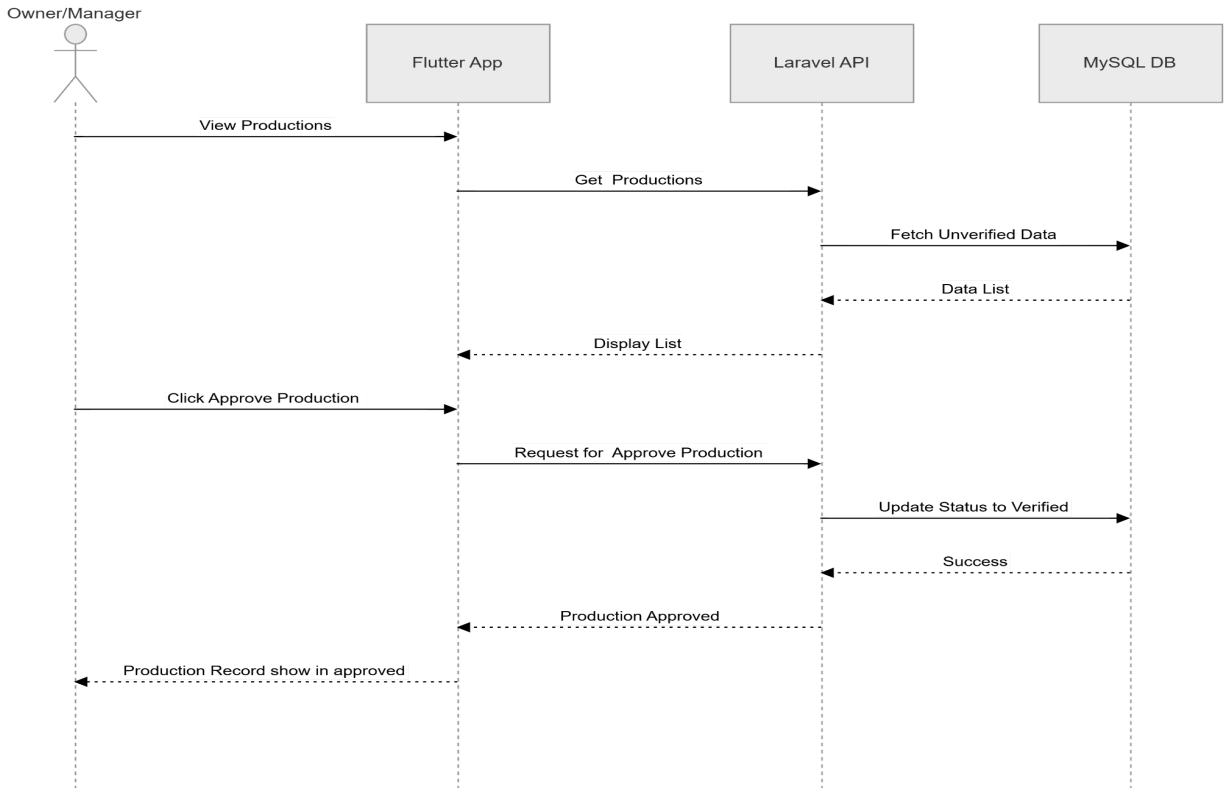
View Machine Detail, Generate QR Code:

Figure 8: Sequence Diagram



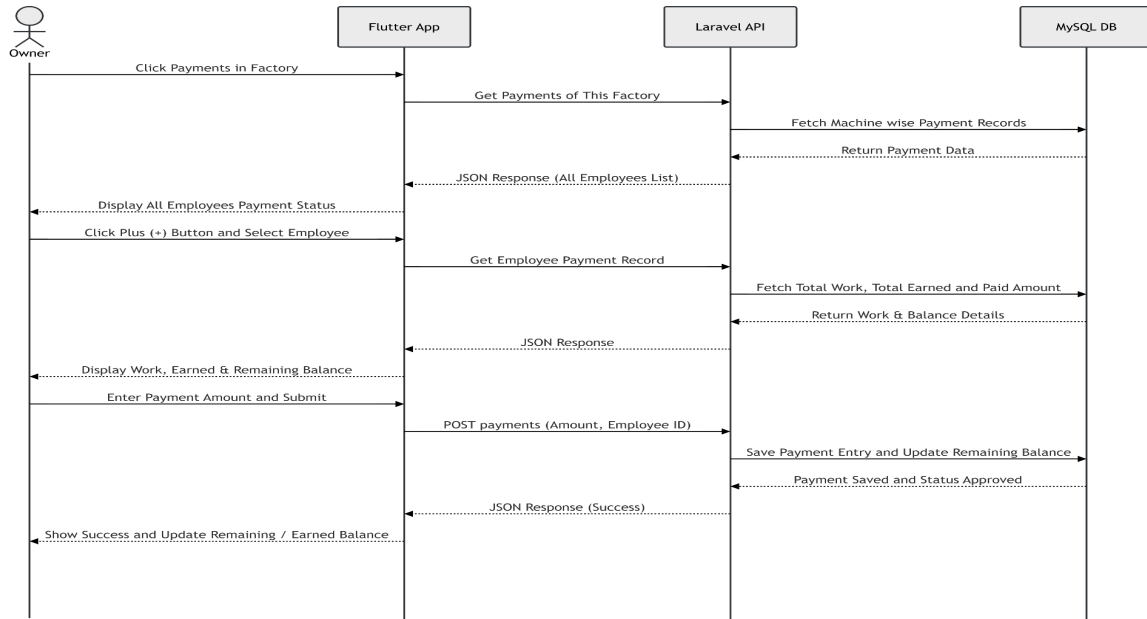
View Productions, Approve Productions:

Figure 9: View Production, Approve Production



View Payments, Approve Payments:

Figure 10: View Payments, Approve Payments

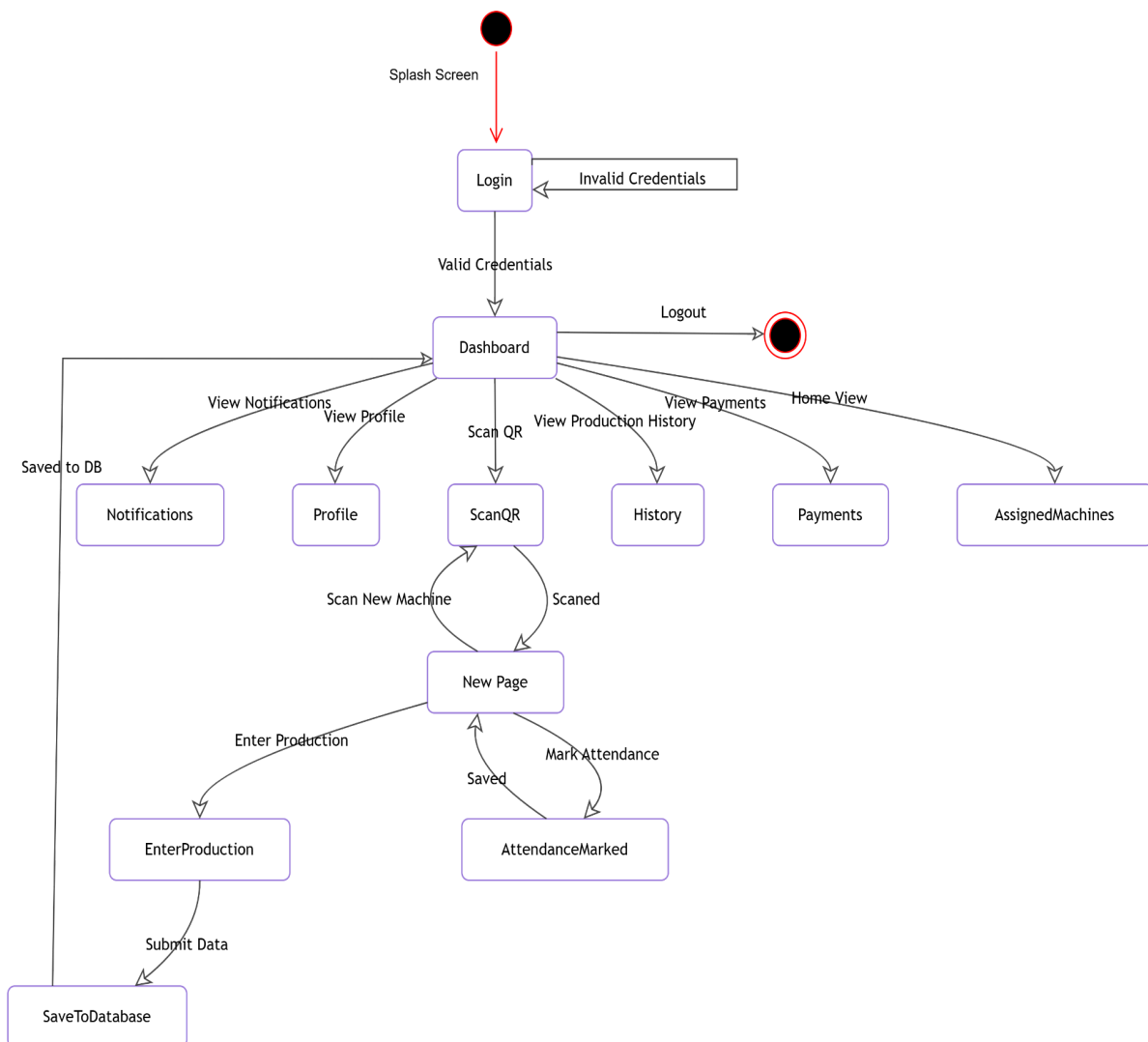


3.2.4 State Transition Diagram

The state transition diagram shows the different states a production entry can be in, such as Pending, Manager Approved, Owner Approved, or Rejected, and how it moves from one state to another.

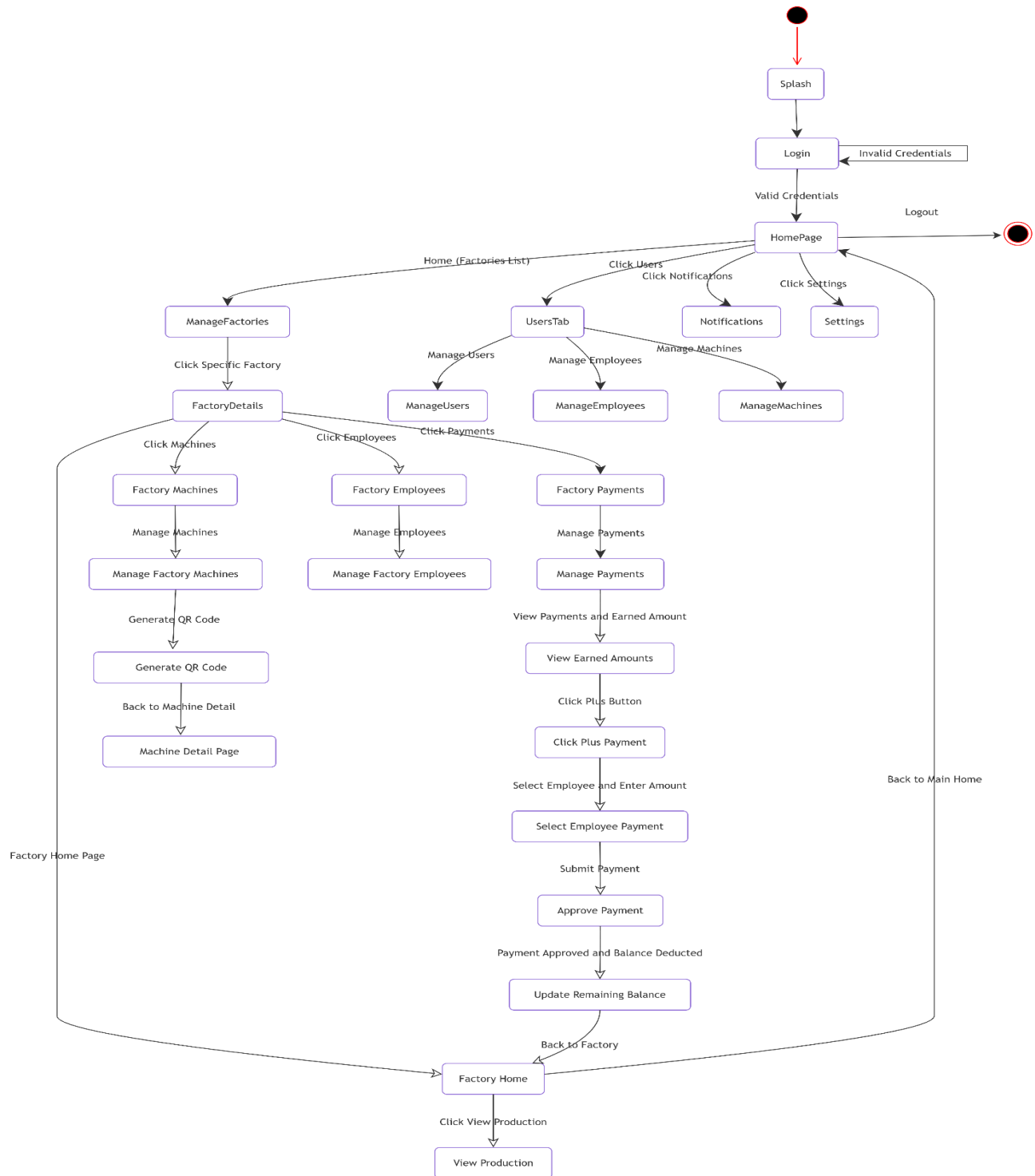
Employee State:

Figure 11: Employee State Transaction Diagram



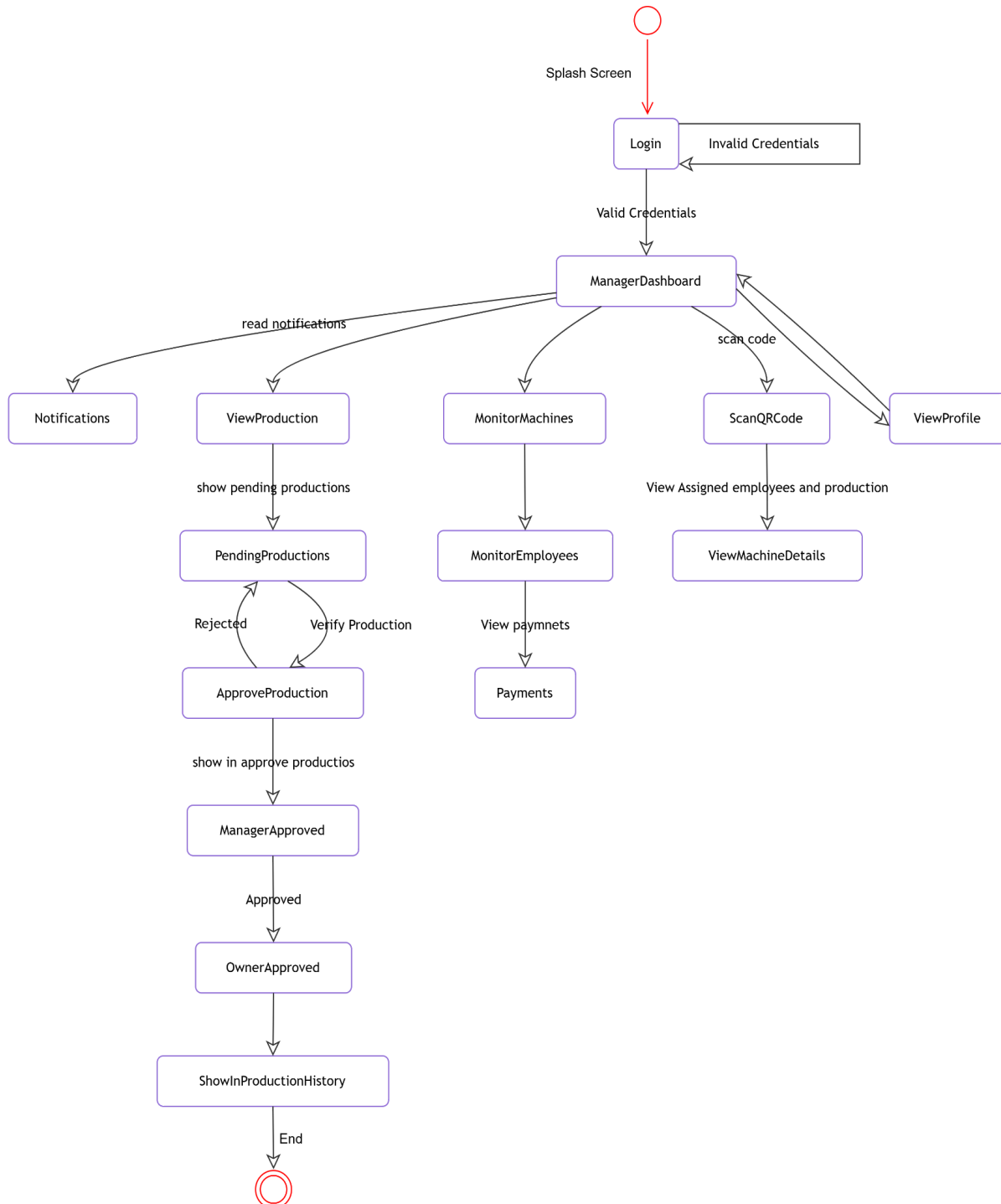
Owner State:

Figure 12: Owner State Sequence Diagram



Manager State:

Figure 13: Manager State Transaction Diagram



3.3 Architecture Diagram

The architecture diagram shows the overall structure of the system, including the Flutter mobile application, the Laravel backend, and the MySQL database, and how data moves between these three layers through REST APIs.

3.3.1 Architecture Diagram

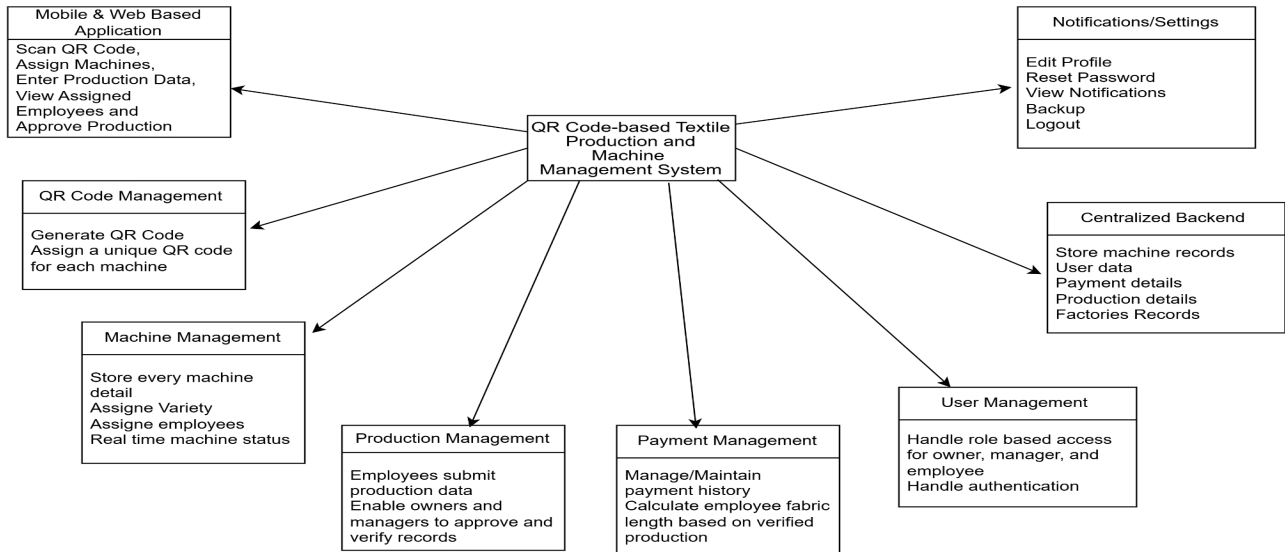
Figure 14: Architecture Diagram



3.3.2 Component Diagram

The component diagram shows the main building blocks of the system, such as the mobile application, the QR module, the backend API, and the database, and how these components talk to each other.

Figure 15: Component Diagram



3.3.3 Layered Architecture (4-Tier)

Table 3-2: Layered Architecture with Responsibilities

Layer	Modules	Responsibilities
Presentation Layer (UI)	Frontend (Flutter)	Handles user interaction, displays owner/manager/employee screens, and provides QR scanning and production entry interface.
Application Layer (Business Logic)	User & Role Management, Machine & Factory Management, Production Verification, Payment Management	Implements core logic such as user authentication, role permissions, machine and factory management, and the manager-owner production approval workflow.
Service Layer (Processing)	Authentication service, QR generation/scanning service, Notification service	Processes QR code generation and scanning, and handles API communication between the mobile app and the Laravel backend.
Data Layer (Storage)	Database (MySQL)	Stores user, machine, production, attendance, payment, notification and backup data.

3.3.4 MVC

Table 3-3: MVC Component Modules Assigned

MVC Component	Modules Assigned
Model (Data & Business Logic)	User, Role, Factory, Machine, Employee, Production, Attendance, Payment, Notification, Backup
View (UI)	User Interface (Flutter Screens)
Controller (Handles Requests)	Handles user authentication, machine and QR operations, production entry and approval, attendance, payment, and backup requests

3.4 Data Design

TECHstile uses a relational database approach (MySQL) to transform the real world factory entities into structured tables of data. MySQL stores data in a well defined plan using tables, rows, and relationships, making it suitable for managing machine records, production entries, employee assignments, verification status, and payment details. Our system is backed by a PHP Laravel backend, which handles data processing, validation, and ensures secure communication between the mobile app and the database. This approach ensures accurate record keeping, efficient data retrieval, data integrity, and scalability of the system. This data model also supports reliable storage, managed updates, and easy integration with the app for managing production, attendance, and payment.

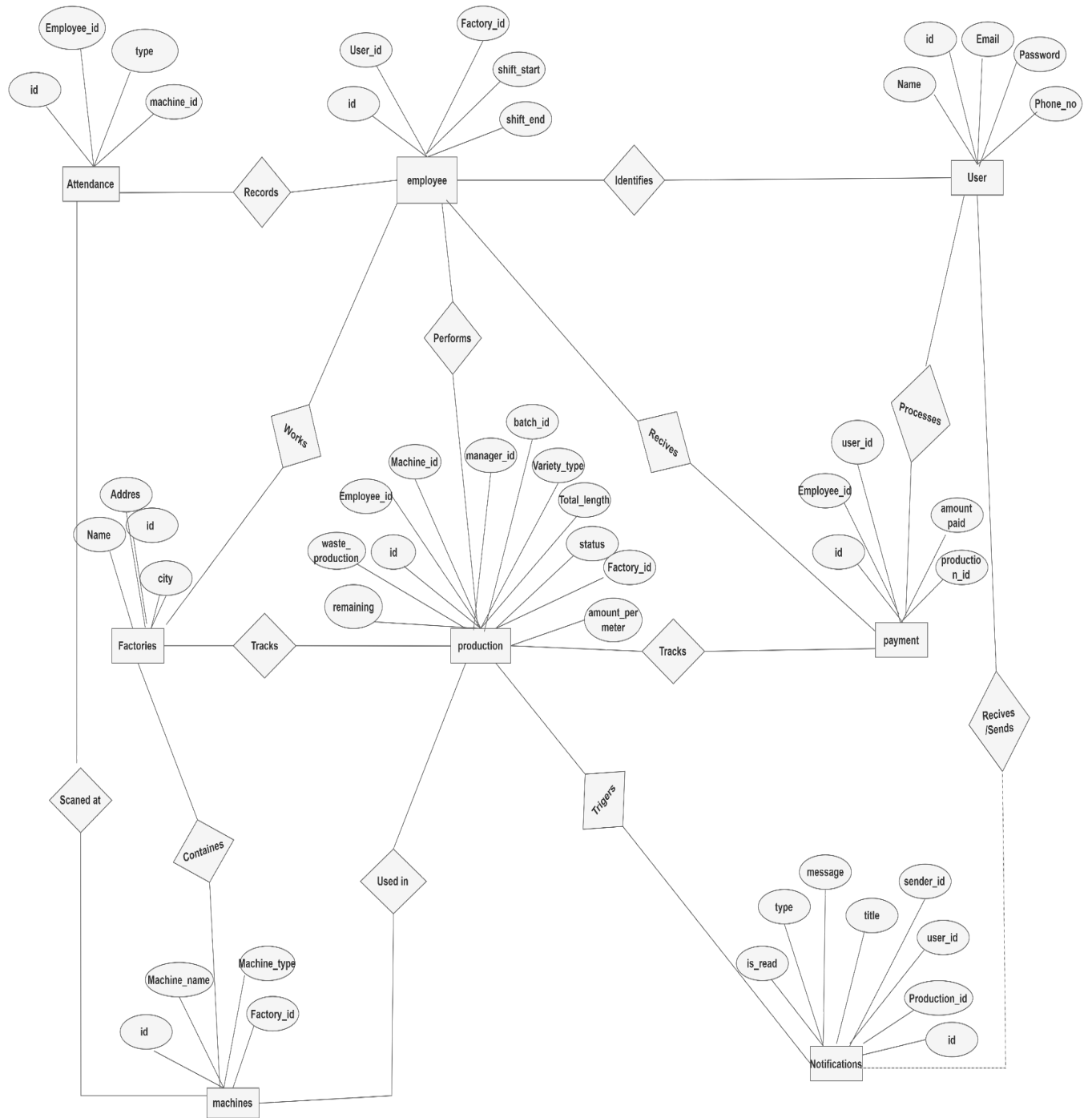
3.4.1 QR Code Data Specification

Table 3-4: QR Code Data Specification

Field	Value	Rationale
Format	Plain text string	Simplest format, no parsing needed on the scanning side
Content	Machine ID	The QR code only carries the machine's unique database ID, nothing else
Encoding	UTF-8 / QR Version	Minimal payload, no sensitive data stored in the code itself
Error Correction Level	M (15%)	Sufficient for short strings, works on slightly damaged or dirty codes
Size when printed	3cm x 3cm minimum	Readable by low end phone cameras on the factory floor

3.4.2 ER Diagram

Figure 16 : ER Diagram



3.4.3 MySQL Data Structure

- **Users Table:** Stores registered users such as Owner, Manager, and Employee Data.
- **Employees Table:** Stores employee specific details linked to a user account, such as shift timing and the factory they belong to.
- **Factories Table:** Stores details of each factory.
- **Machines Table:** Contains machine details under a specific factory. The QR code itself is not stored separately in this table it is generated on the app using the machine's ID.
- **Productions Table:** Maintains daily production entered by employees, along with quantity and approval status.
- **Attendances Table:** Stores employee attendance records.
- **Payments Table:** Stores payment amounts linked to a specific verified production record.
- **Notifications Table:** Stores in app alerts sent to users for actions needing their attention.

3.4.4 Data Processing and Organization

- **Storing Data:** The system stores registration and authentication details of users, machine and factory information, production entries, attendance, and payment records securely in MySQL.
- **Retrieving Data:** Authorized users can retrieve machine details, production records, attendance history, and payment history from the MySQL database based on their role.
- **Processing Data:** The stored data is processed to confirm production entries through manager and owner approval, calculate payments from verified production, update machine and attendance status, generate notifications, and create backups of factory data when triggered by the owner.

3.4.5 Data Dictionary

Table 3-5: Table Data Dictionary

Entity / Attribute / Method	Description
Owner	Object representing the factory owner.
Owner.manageMachines()	Method to add, update or remove machines.
Owner.generateQR()	Method to generate a QR code for a machine.
Owner.ownerAction()	Method used to give final approval or rejection on a production entry.
Owner.manageUsers()	Method to manage managers and employees.
Owner.backupData()	Method to trigger a backup of factory data.
Manager	Object representing the factory manager.
Manager.assignEmployee()	Assigns an employee to a scanned machine.
Manager.managerAction()	Approves or rejects production entered by an employee.

Employee	Object representing the factory worker.
Employee.scanQR()	Scans a machine QR code to access machine details.
Employee.enterProduction()	Enters daily production data against the assigned machine.
Employee.markAttendance()	Marks the employee's own attendance for the day.
Machine	Object representing a factory machine.
Machine.machineId	Unique identifier for each machine.
Machine.machineName	Stores the machine's name.
Machine.factoryId	Links the machine to a specific factory.
Production	Object storing a production entry.
Production.productionId	Unique identifier of the production entry.
Production.readyProduction	Stores the finished production quantity.

Production.wasteProduction	Stores the wasted/rejected quantity.
Production.status	Indicates pending, manager approved, owner approved or rejected state.
Payment	Object representing an employee payment.
Payment.paymentId	Unique identifier for the payment record.
Payment.amountPaid	Calculated payment amount.
Payment.productionId	Links the payment to the verified production it is based on.

3.4.6 Object-Oriented (OO) Approach: Objects, Attributes, Methods & Method Parameters

1. Object: User

Attributes: userId: int, name: string, email: string, password: string, factoryId: int

Methods: login(), logout()

2. Object: Owner

Attributes: role: string

Methods: manageMachines(), generateQR(), ownerAction(productionId), manageUsers(), backupData()

3. Object: Manager

Attributes: assignedFactoryId: int

Methods: assignEmployee(machineId, employeeId), managerAction(productionId)

4. Object: Employee

Attributes: employeeId: int, userId: int, factoryId: int, shiftStartTime: time, shiftEndTime: time

Methods: scanQR(), enterProduction(machineId, quantity), markAttendance()

5. Object: Machine

Attributes: machineId: int, machineName: string, machineType: string, factoryId: int

Methods: create(), update(), delete()

6. Object: Factory

Attributes: factoryId: int, name: string, address: string, city: string

Methods: create(), update()

7. Object: Production

Attributes: productionId: int, machineId: int, employeeId: int, readyProduction: float, wasteProduction: float, status: string

Methods: submitProduction(), managerAction(), ownerAction()

8. Object: Attendance

Attributes: attendanceId: int, employeeId: int, machineId: int, type: string

Methods: markAttendance()

9. Object: Payment

Attributes: paymentId: int, employeeId: int, productionId: int, amountPaid: float

Methods: store(), update()

10. Object: Notification

Attributes: notificationId: int, userId: int, title: string, isRead: boolean

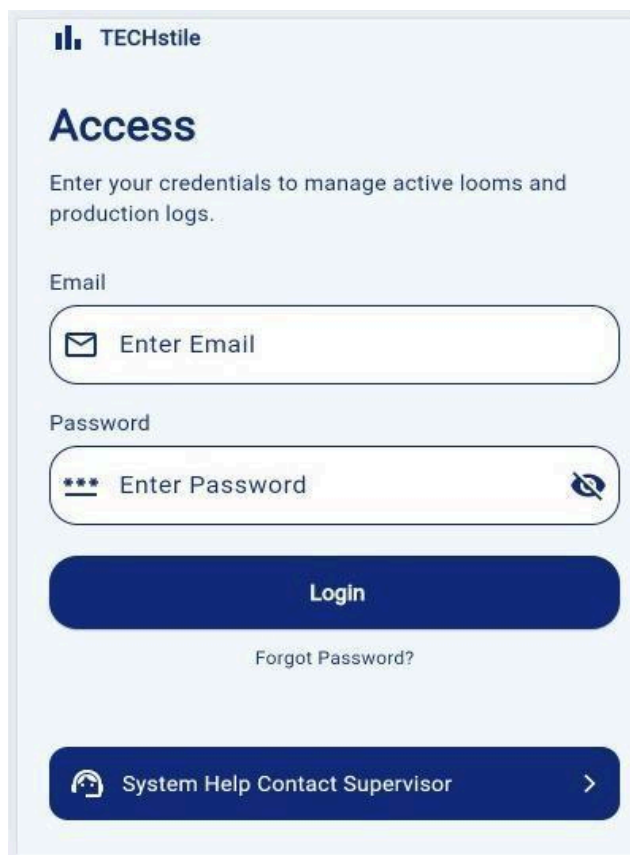
Methods: send(), markAsRead().

3.4.7 User Interface Design

The user interface of our Techstile is designed to be simple and easy to direct for users. It allows Owner, Manager, and Employee users to access the different features of the system according to their roles. The interface helps the users to perform tasks efficiently and correctly.

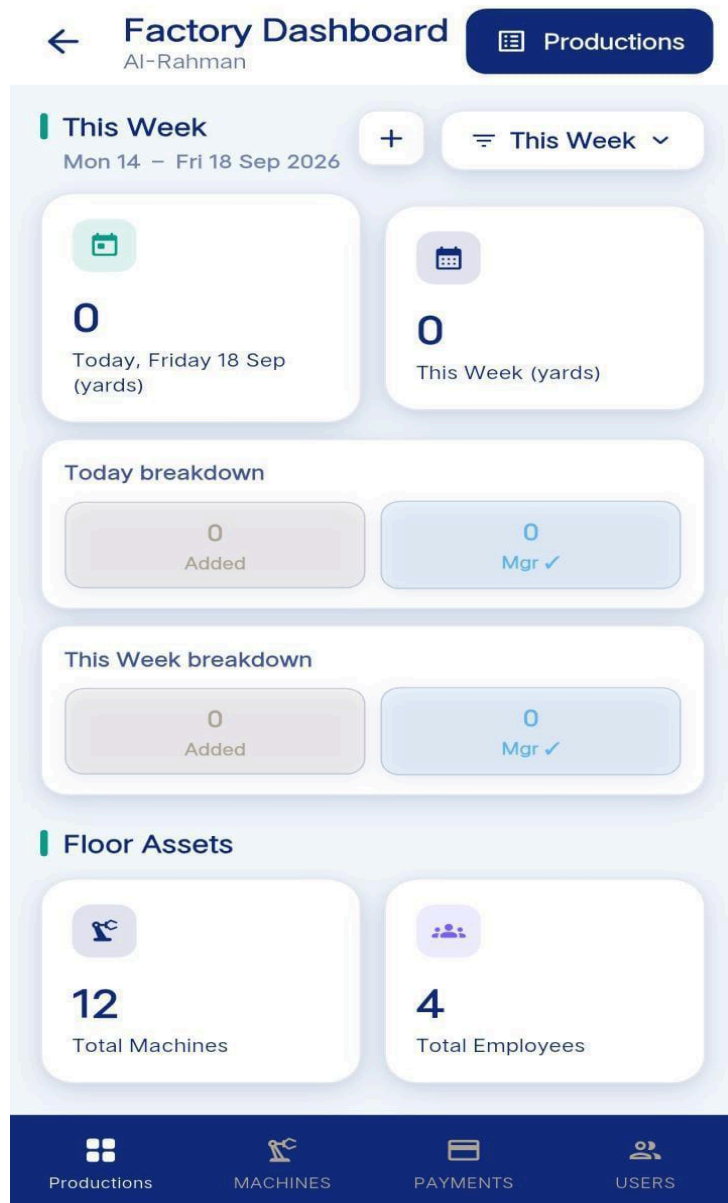
3. Screen Images:

Login Screen:

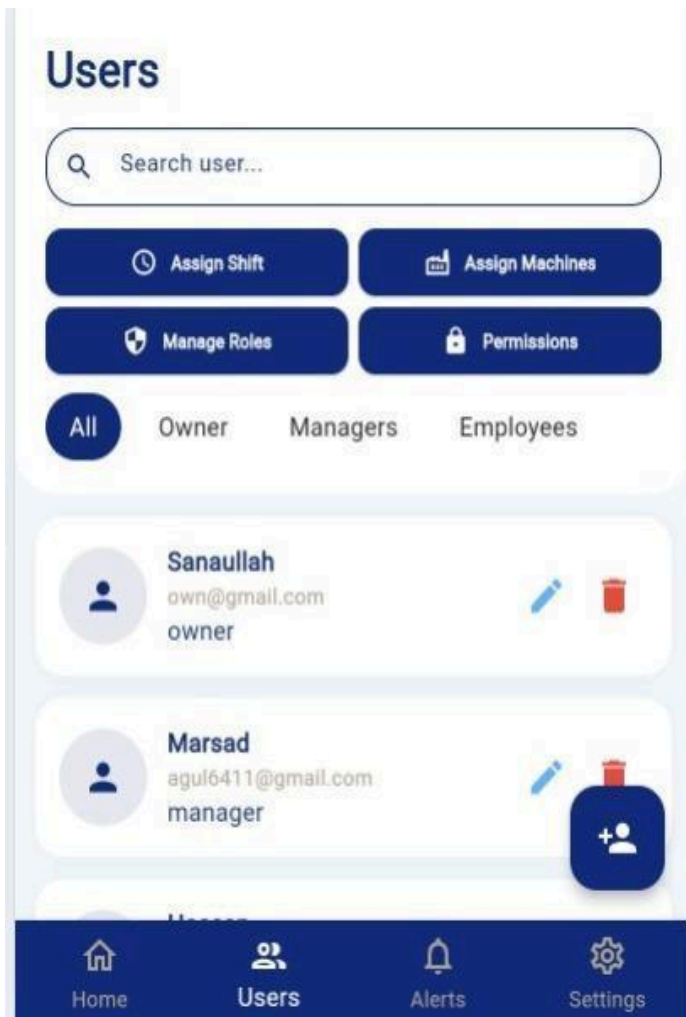


The screenshot displays the login interface for 'TECHstile'. At the top left is the logo, consisting of three vertical bars of increasing height followed by the text 'TECHstile'. Below the logo is the heading 'Access' in a large, bold, dark blue font. Underneath the heading is a sub-heading in a smaller, regular font: 'Enter your credentials to manage active looms and production logs.' The form contains two input fields: an 'Email' field with a light blue border and a white background, containing the placeholder text 'Enter Email' and a small envelope icon; and a 'Password' field with a similar border, containing the placeholder text 'Enter Password', three asterisks, and a small icon of an eye with a slash through it. Below these fields is a large, rounded, dark blue button with the word 'Login' in white text. Underneath the button is a link that says 'Forgot Password?'. At the bottom of the form is another dark blue button with a white icon of a person at a desk, followed by the text 'System Help Contact Supervisor' and a white right-pointing arrow.

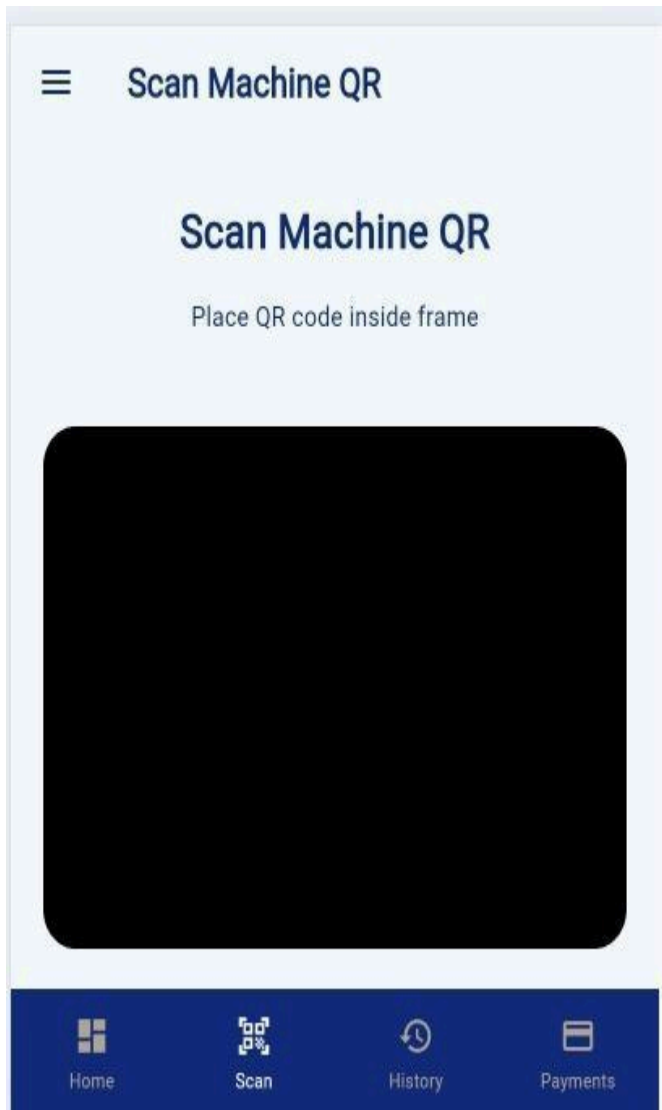
Factory Dashboard Screen:



Manage machines, Users , Shifts,Roles and Permissions:



Scan Machine QR Code Screen:



Machine Detail Page:



MC-01

Neelam Factory

Active

Machine Info

#

Machine ID

1



Machine Type

Power Loom

Shift-wise Production

 Arqam

08:00:00 - 20:00:00

Total

0

Ready

0

Remaining

0

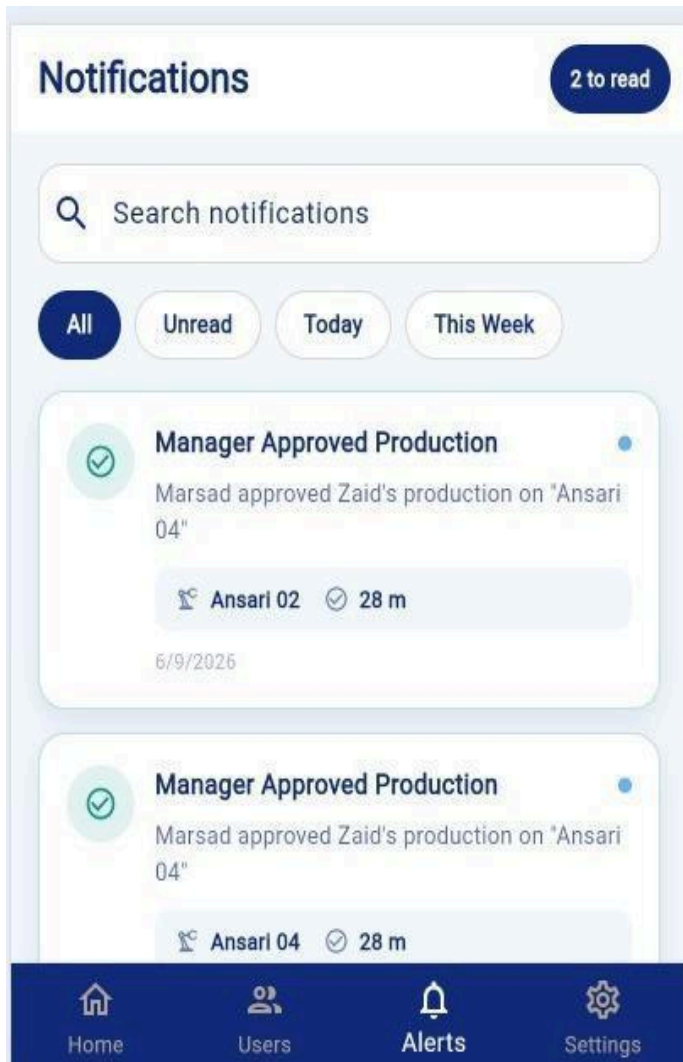
 HOME

 MACHINES

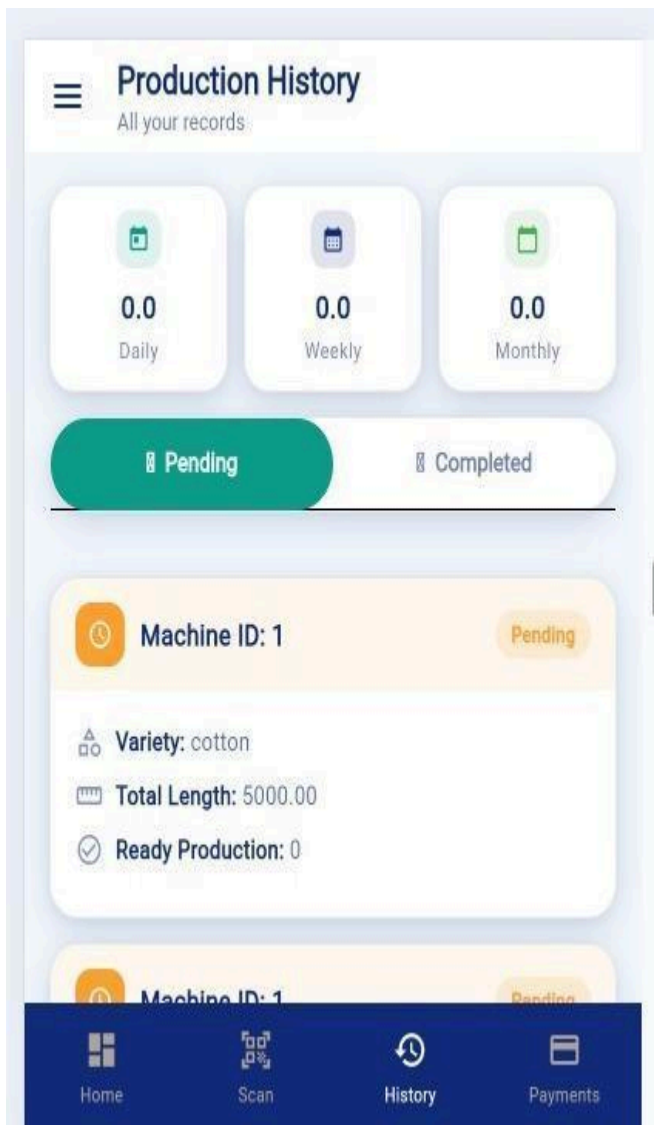
 EMPLOYEES

 PAYMENTS

Notifications Screen:



Production History Screen:



Employee Payments Screen:



3.5 Design Decisions

The designed decisions of the system are made by considering system efficiency, adjustability, security, and long-term durability to support real world textile factory operations.

System Architecture Design

- **Choice:** Client–Server, Layered Architecture with MVC pattern
- **Reason:** This architecture clearly differentiates the presentation layer, business logic, and data layer. The MVC pattern used in the PHP Laravel backend helps in organized code structure, easy maintenance, and smooth combination with QR code processing and the MySQL database.

Data Design

- **Choice:** Relational Database (MySQL)
- **Reason:** MySQL provides structured data storage using tables and relationships, which is suitable for managing users, machines, production records, and payments. It ensures data consistency, integrity, and efficient querying for reports and calculations.

Object Oriented Design Pattern

- **Choice:** Indexed Searching and Filtering
- **Reason:** Indexing in MySQL allows fast searching and filtering of machines, production

Algorithmic Approach

- **Choice:** Indexed Searching and Filtering
- **Reason:** Indexing in MySQL allows fast searching and filtering of machines, production entries, and reports, improving system performance compared to simple linear searches

Security & Authentication

- **Choice:** Role Based Access Control with Secure Authentication
- **Reason:** Restricts system access based on user roles. Admin and Manager can verify production and approve payments, while Employees can only enter and view their own production data.

QR Code Processing

- **Choice:** Unique QR Code Generation and Validation
- **Reason:** Each machine is assigned a unique QR code to enable quick identification, reduce manual errors, and ensure accurate machine wise production tracking.

User Interface Design

- **Choice:** Flutter for Frontend Development
- **Reason:** Flutter provides a responsive and high performance user interface and supports cross-platform mobile deployment, making it suitable for factory environments with different users.

3.6 Summary

This chapter explained how TECHstile is structured and designed. It covered the assumptions the system depends on, its limitations, and the main risks along with how they are handled. It also introduced the design models, context diagram, class diagram, sequence diagram, state transition diagram, component diagram and architecture diagram, that show how the system works internally based on the requirements defined in previous chapter.